

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Магнитогорский государственный технический университет им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И.Носова»)

Институт энергетики и
автоматизированных систем

Кафедра вычислительной техники и
программирования

Направление 09.03.01 –
Информатика и вычислительная
техника

Допустить к защите
Заведующий кафедрой

_____/О. С. Логунова/
«____» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

обучающегося Шемякина Константина Антоновича

на тему: «Разработка программной библиотеки для осуществления прямого управления
эмбоссером MATICA через TranzWare CardFactory»

ВКР выполнена на __ страницах

Графическая часть на __ листах

Руководитель: _____ доцент кафедры ВТ и П, к.т.н. Калитаев А. Н.
(подпись, дата, должность, ученая степень, ученое звание, Ф.И.О)

Нормоконтроль и проверка
на антиплагиат выполнены.

Оригинальность текста __ %

_____/А. Н. Калитаев/
(подпись, дата)

Обучающийся _____
(подпись)

«____» _____ 20 ____ г.

Магнитогорск 2025

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Магнитогорский государственный технический университет им. Г.И. Носова»
(ФГБОУ ВО «МГТУ им. Г.И.Носова»)

Кафедра вычислительной техники и
программирования

УТВЕРЖДАЮ:

Заведующий кафедрой

_____ /О. С. Логунова/

«_____» _____ 2025 г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

ЗАДАНИЕ

Тема: «Разработка программной библиотеки для осуществления прямого управления
эмбоссером MATICA через TranzWare CardFactory»

Обучающемуся Шемякину Константину Антоновичу

Тема утверждена приказом № 10-35/376 от 27.02.2025 г.

Срок выполнения «19» июня 2025 г.

Исходные данные к работе:

- 1) ГОСТ 19.701-90 Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.
- 2) Техническое задание в системе JIRA
- 3) Техническая документация о платформе FloraWare.
- 4) Техническая документация о системе TranzWare CardFactory.
- 5) Техническая документация о приложении Maticard PRO.
- 6) Техническая документация об эмбоссерах MATICA.
- 7) СМК-О-СМГТУ-36-20 «Выпускная квалификационная работа: структура, содержание, общие правила выполнения и оформления»

Перечень вопросов, подлежащих разработке в выпускной квалификационной работе:

- 1) Теоретический анализ кардридерного модуля эмбоссеров MATICA и сопутствующего ПО для обработки пластиковых карт.
- 2) Проектирование и разработка кардридерного драйвера для включения в продукт TranzWare CardFactory.
- 3) Внедрение кардридерного драйвера в TranzWare CardFactory и тестирование готового программного решения.

Графическая часть:

1. Титульный лист.
2. Компоненты эмбоссера
3. Анализ текущих решений
4. Цель, объект и предмет исследования
5. Задачи исследования
6. Структурная схема ПО TranzWare CardFactory
7. Схема сетевого подключения к модулю
8. Схема взаимодействия с модулем
9. Проектное решение модуля обработки
10. Проектное решение драйвера
11. Полученный результат
12. Преимущества программного решения

Руководитель: _____ / А. Н. Калитаев /

(подпись, дата)

Задание получил: _____ / К.А. Шемякин /

(подпись, дата)

ОТЗЫВ

на выпускную квалификационную работу студента гр. АВб-21-12

Института энергетики а автоматизированных систем

ФГБОУ ВО «МГТУ им. Г.И. Носова»

Шемякина Константина Антоновича

на тему «Разработка программной библиотеки для осуществления

прямого управления эмбоссером MATICA через TranzWare

CardFactory»

Целью выпускной квалификационной работы является создание программной библиотеки внутри программного продукта компании «Компас плюс» TranzWare CardFactory, посредством использования функций которой будет осуществляться прямое управление кардридерным модулем эмбоссера MATICA и, впоследствии, запись данных на чип пластиковой карты.

Работа выполнена в соответствии с заданием и отличается грамотностью, логичностью и полнотой изложения, что подтверждает глубину проработки материала студентом Шемякиным К.А., а также его усердие и ответственность при написании выпускной квалификационной работы.

В первой главе выпускной квалификационной работы рассмотрены ключевые понятия процесса эмбоссирования и записи чипа пластиковой карты. Проанализированы компоненты эмбоссера MATICA и его кардридерного модуля. Рассмотрены внутренняя структура программного обеспечения для обработки банковских карт и текущие решения по персонализации пластиковых карт с использованием кардридерного модуля эмбоссера MATICA. А также обозначены и сформулированы цель и задачи выбранного исследования.

Во второй главе работы рассмотрены проектные решения по разработке программной библиотеки и ее интеграции в программный продукт. Был

выведен полный список требований к программной библиотеке. Построены структурные схемы программного решения и сопутствующих процессов. Описаны использующиеся инструментальные средства разработки в соответствии с техническим заданием. Представлена модель, отражающая планирующиеся интерфейсные доработки.

В третьей главе описано разработанное программное решение библиотеки. Рассмотрена архитектура библиотеки и подробно описаны все элементы объектного дерева библиотеки. Представлены и подробно описаны коды всех функций программной библиотеки. Описан порядок работы с программным продуктом TranzWare CardFactory для обработки пластиковой карты средствами кардридерного модуля эмбоссера.

Считаю, что выпускная квалификационная работа, выполненная Шемякиным К.А., безусловно, заслуживает оценки «отлично», а выпускник – присвоения квалификации «бакалавр» по направлению «Информатика и вычислительная техника».

Научный руководитель:

доцент кафедры вычислительной

техники и программирования,

канд. тех. наук

_____ /А.Н. Калитаев/

РЕФЕРАТ

Всего ВКР выполнена на 87 листах. В работе содержится 3 иллюстрации спроектированных функциональных схем, отражающих положение будущего программного решения и одна иллюстрация функциональной схемы процесса обработки карты средствами программного продукта TranzWare CardFactory. Всего работа содержит 4 приложения, содержащих листинги кодов разработанных функций программной библиотеки.

Эмбоссер, эмбоссер MATICA, TranzWare CardFactory, кардридер, кардридерный модуль эмбоссера, чип пластиковой карты, кардридерный драйвер, драйвер записи карты, Chip Writer Driver, интерфейс модуля настроек

Объектом исследования работы послужил процесс записи данных на чип пластиковой карты средствами TranzWare CardFactory с использованием кардридерного модуля эмбоссеров MATICA. Целью работы является создание дополнительного кардридерного драйвера для модуля обработки Chip Writer Driver внутри продукта TranzWare CardFactory, который предоставит все необходимые функции для взаимодействия с кардридерным модулем эмбоссера. При выполнении работы проводилось исследование всех необходимых компонентов посредством изучения внутренней документации компании, выполнялось проектирование будущего проектного решения и планирование изменений к внесению в TranzWare CardFactory для интеграции будущего решения. Отличительной особенностью разработки является обеспечение взаимодействия с аппаратным интерфейсом кардридерного модуля эмбоссера программными средствами платформы FloraWare. Ранее, процесс персонализации карты с использованием эмбоссеров MATICA проходил в два этапа, а именно этапы эмбоширования и записи чипа. Разработка драйвера направлена на сокращение времени между этапами, объединении этапов в один непрерывный процесс при персонализации средствами TranzWare CardFactory, создание ситуации отсутствия надобности

переноса карт между этапами, что уменьшит присутствие человеческого фактора в процессе персонализации. Результатом работы является внедренный в TranzWare CardFactory кардридерный драйвер. Все результаты работы были внесены в актуальную версию продукта CardFactory и уже используются в процессинговых центрах Нигерии для выпуска пластиковых банковских карт с использованием эмбоссеров компании MATICA. В дальнейшем, при разработке компанией MATICA новых кардридерных модулей или по требованию клиентов, планируется доработка драйвера для добавления возможности внесения дополнительных настроек, поддержки работы с новыми режимами записи карты и т.д.

СОДЕРЖАНИЕ

Введение.....	10
1 Теоретические и методологические основы работы с эмбоссерами и сопутствующего программного обеспечения	13
1.1 Анализ эмбоссера MATICA.....	13
1.1.1 История возникновения эмбоссеров	13
1.1.2 Общее описание работы эмбоссеров	16
1.1.3 Теоретические основы работы с кардридерным модулем эмбоссеров MATICA	18
1.2 Анализ программного обеспечения для осуществления персонализации средствами эмбоссеров MATICA.....	19
1.2.1 Платформа FloraWare	20
1.2.2 Программный продукт TranzWare CardFactory	24
1.2.3 Утилита Maticard PRO	28
1.3 Анализ текущих решений.....	30
1.3.1 Анализ кардридерных драйверов продукта TranzWare CardFactory	30
1.3.2 Анализ актуального процесса персонализации карт с использованием эмбоссеров MATICA.....	32
1.4 Постановка цели и задач	34
1.5 Выводы	36
2 Проектные решения для разработки драйвера прямого управления эмбоссером MATICA средствами FloraWare	39
2.1 Описание функциональных требований, предъявляемых к разрабатываемому драйверу	39

2.2 Функциональная схема разрабатываемого драйвера	41
2.2.1 Общее описание функциональных схем.....	45
2.2.2 Поток данных и управления.....	46
2.2.3 Альтернативные сценарии	48
2.3 Описание используемых инструментальных средств разработки.....	49
2.4 Проектирование пользовательского интерфейса.....	55
2.4.1 Общие принципы проектирования интерфейса.....	56
2.4.2 Планирование интерфейсных доработок	56
2.4.3 Визуализация интерфейса	58
2.5 Выводы	60
3 Описание и порядок работы с разработанным программным обеспечением	62
3.1 Структура драйвера.....	62
3.1.1 Архитектура драйвера	62
3.1.2 Объектное дерево драйвера.....	66
3.2 Порядок работы с разработанным кардридерным драйвером	75
3.2.1 Порядок настройки карточного продукта	76
3.2.2 Порядок запуска обработки	83
3.3 Выводы	86
Заключение	87
Список использованных источников	90
Приложение А Листинг кода объекта-функции Open.....	91
Приложение Б Листинги кодов объектов-функций WaitCard, Restart и CardConnect.....	92
Приложение В Листинги кодов объектов-функций TCPSend и Transfer.....	93
Приложение Г Листинги кодов объектов-функций EjectCard и CardClose	95

ВВЕДЕНИЕ

Современный финансовый сектор, характеризующийся динамичным развитием и непрерывной гонкой за лидерство, требует от банков постоянного совершенствования операционной деятельности и оптимизации бизнес-процессов. В условиях жесткой конкуренции, где качество обслуживания и скорость предоставления услуг становятся ключевыми факторами успеха, банки стремятся минимизировать издержки, повысить производительность и, что самое главное, удовлетворить растущие потребности клиентов. Одним из важнейших элементов в цепочке оказания банковских услуг является процесс выпуска пластиковых карт, который включает в себя множество этапов, от разработки дизайна до персонализации и выдачи готового продукта клиенту [9].

На этапе персонализации, когда на карту наносится индивидуальная информация, ключевую роль играет эмбоссер – специализированное устройство, предназначенное для рельефного нанесения символов, таких как имя держателя карты, номер карты, срок действия и т.д. Эмбоссирование является неотъемлемой частью процесса выпуска карт и обеспечивает их уникальность и безопасность, а также позволяет проводить транзакции в тех терминалах, которые не поддерживают электронное считывание. Несмотря на появление более современных технологий персонализации, таких как печать, эмбоссирование по-прежнему широко используется, особенно в сегменте премиальных карт, где рельефное нанесение информации подчеркивает их статусность и обеспечивает тактильные ощущения при использовании. Таким образом, от эффективности работы эмбоссеров напрямую зависит не только пропускная способность всего производственного цикла, но и скорость предоставления карт клиентам, что, в свою очередь, оказывает непосредственное влияние на удовлетворенность клиентов и репутацию банка.

Однако, на практике, работа эмбоссеров не всегда соответствует требуемым стандартам эффективности. Низкая производительность, частые сбои в работе, необходимость проведения регулярного технического обслуживания и замены расходных материалов являются серьезными препятствиями, препятствующими эффективному процессу персонализации. Кроме того, недостаточное внимание к технологическим аспектам работы эмбоссеров, а также устаревшие методы их эксплуатации могут приводить к увеличению времени обработки карт, повышению затрат и снижению качества готовой продукции. Учитывая все вышесказанное, исследование путей повышения эффективности работы эмбоссеров для банковских карт является актуальной и практически значимой задачей, решение которой позволит банкам оптимизировать свои операционные процессы, повысить качество предоставляемых услуг и укрепить свои позиции на рынке.

Российская компания «Компас плюс» является лидером в IT-сфере банковского обслуживания. Она предоставляет банкам решения для управления банкоматами и терминалами самообслуживания, фронт-менеджмента, обработки цифровых платежей, торгового эквайринга и т.д. Также, немаловажной отраслью деятельности компании является поддержка эмиссии и персонализации карточных продуктов, которая включает в себя эмбоссирование физической карты. Так как «Компас плюс» охватывает большую долю мирового рынка банковской IT-сферы, для нее необходимо, чтобы выпускаемое ПО работало быстро, стабильно и поддерживало работу с самым современным оборудованием.

Компания «Компас плюс» работает с многими крупными производителями эмбоссеров по всему миру, получая от них документацию для поддержки работы эмбоссеров, используемых клиентами. Среди таких компаний, итальянская компания MATICA является одним из крупнейших производителей эмбоссеров и на данный момент занимает значимую долю рынка.

Целью данной дипломной работы является автоматизация записи информации на чип пластиковой банковской карты средствами TranzWare

CardFactory с использованием эмбоссеров компании MATICA. Для реализации поставленной цели необходимо решение следующих задач:

- Выявить структуру кардридерного модуля эмбоссера MATICA и способы программного и аппаратного взаимодействия с ним, а также выявить структуру и возможности сопутствующего программного обеспечения, используемого для обработки пластиковых карт;
- Разработать проектное решение нового кардридерного драйвера для программного продукта TranzWare CardFactory, который будет осуществлять прямое управление кардридерным модулем эмбоссера;
- Описать внедренное в TranzWare CardFactory программное решение кардридерного драйвера и порядок работы с ним в тестовом сценарии;

Объектом исследования являются процесс записи информации на чип пластиковой карты средствами программного продукта TranzWare CardFactory и сопутствующего программного обеспечения для работы с эмбоссерами с использованием кардридерного модуля эмбоссера MATICA.

Предметом исследования является анализ структуры кардридерного модуля эмбоссера MATICA, методов аппаратного и программного взаимодействия с ним и сопутствующего программного обеспечения для персонализации пластиковых банковских карт.

Данная дипломная работа состоит из введения, трех глав, заключения и списка литературы. Первая глава посвящена теоретическим основам процесса эмбоссирования и анализу существующих технологий. Вторая глава содержит описание этапов проектирования будущего программного решения драйвера. В третьей главе проводится полный разбор разработанного драйвера управления кардридерным модулем и порядок работы с разработанным драйвером для пользователей. В заключении представлены основные выводы и результаты исследования.

1 ТЕОРЕТИЧЕСКИЕ И МЕТОДОЛОГИЧЕСКИЕ ОСНОВЫ РАБОТЫ С ЭМБОССЕРАМИ И СОПУТСТВУЮЩЕГО ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1 Анализ эмбоссера MATICA

Прежде чем приступать к планированию и разработке любых проектных решений необходимо досконально проанализировать компоненты эмбоссера, с которыми и посредством которых необходимо будет взаимодействовать с ним. Анализ является необходимой частью, с которой начинается любое исследование, разработка и любая другая большая работа. Это вдвойне справедливо при осуществлении работы с любыми сложными техническими комплексами, такими как, например, эмбоссер, ведь необходимо проанализировать все методы взаимодействия с этим комплексом, порядок работы с ним, его составляющие и т.д. для того, чтобы не упустить любые важные детали, которые могут пригодиться в дальнейшей разработке.

1.1.1 История возникновения эмбоссеров

Перед тем как приступать к анализу такого современного технического комплекса, как эмбоссер, необходимо рассмотреть его в историческом контексте для понимания тех потребностей и процессов, которые привели к появлению данного комплекса на свет в том виде, в котором мы можем наблюдать его на сегодняшний день.

Начнем издалека, когда такого понятия как эмбоссер еще не существовало и о платежных картах не было речи. У истоков платежных карт стояли американские курьерские компании, которые с 1841 года осуществляли перевозки грузов, людей, а также денег и ценностей. Таким образом осуществлялся перевод денег между банками на протяжении нескольких десятилетий. В определенный момент владельцы курьерских компаний осознали насколько небезопасно переводить деньги подобным образом, ведь повозки с деньгами и ценностями часто подвергались грабежу. В те времена был изобретен телеграф, который навел на мысль компанию Wells Fargo, самую популярную на тот момент курьерскую компанию, для осуществления

денежных переводов через него с 1864 года, ведь никто не сможет украсть деньги, которые идут по телеграфным линиям.

В те времена изобретения и открытия стремительно меняли жизнь общества. Развитие науки и техники, активная миграция населения, переезды в поисках работы и развитие торговли обусловили появление платежных карт, дорожных чеков и почтовых переводов. Дорожные чеки являются прародителями платежных карт и были великим изобретением для своего времени. Изобретателем чеков является Томас Кук, основатель и владелец одноименного бюро путешествий в Лондоне. В 1868 году он начал продавать своим клиентам купоны для расчетов в гостиницах Европы (Hotel Coupons). Через шесть лет, в 1874 году их заменили циркулярные аккредитивные письма (Circular Notes), которые служили как для оплаты, так и для получения наличных в местной валюте в партнерских банках и гостиничных обменных пунктах. Транспортная и курьерская компания American Express ввела дорожные чеки в 1891 г. Это случилось после того, как ее основатель Джеймс Фарго в 1890 г. отправился путешествовать по Европе и столкнулся с трудностями при получении наличных по своим американским банковским аккредитивам. Поскольку тогда многие американцы ездили в Европу, он поручил своему специалисту Марселлусу Берри разработать бланк дорожного чека. 7 июля 1891 г. Берри получил патент на новую услугу, которую назвал дорожным чеком (travellers cheque). Все такие чеки и циркуляры были выполнены из бумаги или картона и данные клиента или писались на них чернилами или же выдавливались тиснением.

Более похожие на современные платежные карты предшественники появились в 1870 году. Первым предшественником современных платежных карт считают металлические кредитные жетоны. Номер и логотип магазина выдавливались на них прессами или же выстукивались специальными штампами. Они использовались для покупки клиентами магазинов различных товаров в кредит. Вместо того, чтобы запоминать каждого клиента, магазин выдавал ему номерной кредитный жетон. При покупке с жетоном, его номер записывался в учетную книжку, по записям в которой можно было проследить

все покупки и задолженности. Клиенты должны были обязательно в конце месяца наличными рассчитаться по задолженностям в учетной книге. Такие кредитные жетоны были доступны только людям, которые были в состоянии совершать большие покупки в магазинах, остальным же жетоны не выдавались. В 1924 году появились уже более похожие на карты экземпляры. Компания Mobil Oil предлагала кредитные карты постоянного клиента, которые представляли из себя уже металлические карточки, на которых были выбиты номер карты, имя и адрес клиента.

Первые пластиковые кредитные карты были выпущены компанией American Express в 1959 году. До этого они были бумажными и фиолетового цвета, как и дорожные чеки American Express, но через год, из соображений безопасности материал карт был изменен на пластик. Информация на них выдавливалась горячим прессом с вставками для необходимых букв и цифр. Также появление пластиковых карт ускорило процесс покупки в магазинах, ведь вместе с ними компания выпустила специальные импринтеры или же валидаторы, которые позволяли отпечатывать выдавленные данные с карты на лист бумаги. По сравнению с предыдущими методами, это значительно ускорило процесс покупки, отчего такие карты в свое время пользовались большим спросом. Информация на таких картах выдавливалась с помощью специального прессы, под который подкладывалась карта. Эти прессы были первыми механическими эмбоссерами.

Технический прогресс не стоял на месте и на картах с 1968 года на банковские карты стали наносить магнитную полосу методом горячей штамповки. На полосу записывались данные клиента методом магнитной записи. Магнитная запись была изобретена еще в 1878 году и только через 20 лет была разработана магнитная полоса, которая первоначально служила для записи телефонных разговоров. Разработала технологию записи необходимой информации на магнитную полосу разработала компания IBM. Указание на разработку было дано еще в 1966 году советом директоров. Целью того заседания была разработка технического решения для будущего предложения платежных карт. Одним из первых в 1972 г. поддержал введение магнитной

полосы на картах президент National BankAmericard Ди Хок. Он считал новую технологию дешевой и надежной. Американская банковская ассоциация тоже поддержала магнитную полосу, установила структуру записи на двух дорожках и в 1973 г. сделала полосу своим стандартом. К апрелю 1973 г. 85 % банковских карт уже имело магнитную полосу. С появлением магнитной полосы необходимо было автоматизировать еще один из этапов выпуска карты, поэтому был создан дополнительный агрегат, который записывал информацию на магнитную полосу карты и ламинировал ее. В то же время, в эпоху компьютеризации, ближе к середине 1980-х годов, был разработан автоматический эмбоссер, который производил полноценную персонализацию карты, от эмбоссирования до записи информации на магнитную ленту. Данная разработка была вызвана большим расширением карточной клиентуры, так как американские банки расширялись в европе, китае и японии, где их кредитные карты стали очень популярными среди населения. Такой эмбоссер позже модифицировался после выпуска карт со встроенным чипом, а также под влиянием разработок в области техники и компьютеризации [1].

1.1.2 Общее описание работы эмбоссеров

Как была упомянуто в начале данной главы, эмбоссер представляет из себя сложный технический комплекс. Он состоит из нескольких модулей, каждый из которых выполняет определенную функцию. Модули не имеют определенных названий, так что в рамках работы они будут обозначены по выполняемым функциям. Весь процесс прохождения пластиковой банковской карты через эмбоссер и подстройки ее под клиента называется персонализацией. Весь процесс персонализации в эмбоссере можно разбить на два этапа: запись информации на чип карты и непосредственно эмбоссирование. Оба этапа должны идти непосредственно друг за другом, запись информации никогда не производится посреди процесса эмбоссирования. Между всеми этапами и модулями карта перемещается по специальным салазкам с прорезиненными краями для хорошего сцепления с картой.

Весь этап запись информации производится в одном модуле – кардридерном. Карта помещается туда с салазок специальным манипулятором, который заталкивает ее в кардридер. Внутри кардридера присутствует набор металлических контактов, которые прижимаются к контактной площадке чипа карты и обеспечивают надежное соединение с чипом карты.

Этап эмбоосирования же выполняется несколькими последовательными модулями, которые могут идти в различной последовательности в зависимости от требований к выпуску карты и строения самого эмбоссера. Этапы которые должна обязательно пройти пластиковая карта для завершения эмбоосирования это:

- Печать изображений на поверхностях карты специальными красками
- Выдавливание, выжигание или печать информации о клиенте, номера карты, и CVC/CVV
- Ламинирование карты

В начале этапа эмбоосирования, карта поступает на салазки абсолютно белая и по прохождению всех необходимых модулей приобретает тот вид, в котором мы привыкли ее видеть.

Стоит упомянуть, что хотя эмбоссер и является сложным техническим комплексом, он не предоставляет абсолютно всех средств для осуществления полноценной работы с ним. Для точного управления такой системой необходимо иметь ЭВМ, на которой должно быть установлено необходимое ПО для работы с эмбоссером. Такая управляющая ЭВМ в совокупности с установленными программными средствами позволяет полностью управлять процессом эмбоосирования и посредством внесения предварительных настроек определять вид будущей карты.

Эмбоссеры компании MATICA не являются исключением из тех общих положений, описанных в данном подразделе. Конечно, так как для осуществления разработки не был предоставлен полноценный эмбоссер с документацией, а только его кардридерный модуль для осуществления поддержки прямого управления. В ходе работы были исследованы и раскрыты

только методы взаимодействия именно с ним и в дальнейшем работа с эмбоссером будет рассматриваться только в контексте взаимодействия с его кардридерным модулем.

1.1.3 Теоретические основы работы с кардридерным модулем эмбоссеров MATICA

Предоставленный для разработки и тестирования кардридерный модуль представляет из себя электронную плату на микроконтроллерной основе с подключенными к ней необходимыми периферийными устройствами. Всего в модуль входит три основных компонента: блок питания, управляющая плата и кардридеры. Все компоненты соединены между собой кабелями и проводами. Блок питания подключается к управляющей плате тремя проводами для питания, а кардридеры подключены к управляющей плате через специальную дополнительную плату, которая устанавливается поверх управляющей платы, с несколькими коннекторами, к которым собственно и подключаются ридеры. В модуле может присутствовать любое количество кардридеров, это зависит от модели выпускаемой платы. В тестовой же плате присутствует два кардридера, чего вполне достаточно для полноценной разработки и тестирования разрабатываемого драйвера.

Для соединения с управляющей ЭВМ на плате предусмотрен порт Ethernet. Сама плата должна подключаться к ЭВМ через маршрутизатор и принимать сообщения через внутреннюю локальную сеть или же подключаться напрямую через Ethernet кабель, хотя производителем рекомендуется именно последний вариант подключения. Используя предоставленную документацию модуля, было выявлено, что любое взаимодействие с модулем необходимо осуществлять по протоколу TCP/IP. Любые команды обязательно нужно передавать в кардридерный модуль только по зарезервированным в нем IP-адресам. Модуль обязательно имеет адрес управляющей платы, по которому можно обращаться только к ней, в данном случае его называют базовым адресом. Каждый подключенный кардридер также имеет свой собственный зарезервированный адрес, его можно вычислить путем добавления номера ридера к младшему байту

базового адреса платы. Адреса ридеров используются для осуществления управления ими, которое включает в себя настройку ридеров, запись данных на вставленную пластиковую карту и считывание данных с карты. Стоит упомянуть, что номера кардридеров начинаются с единицы и нумерация ридеров идет слева направо, учитывая только расположение коннекторов, к которым подключены ридеры.

Все команды модулю в обязательном порядке необходимо передавать в виде набора символов, оканчивающегося символом конца последовательности. В наборе обязательно должны содержаться элементы кода на специализированном языке программирования TCPScript. Данный язык был разработан компанией MATICA для использования в своих эмбоссерах. Язык имеет C-подобный синтаксис и обширную документацию, предоставляемую все той же компанией MATICA. При получении строки текста по сети, управляющий модуль анализирует код, находящийся в ней, и выполняет его. Если команды отправляются по адресу одного из ридеров модуля, то код все равно выполняется управляющей платой, если же в коде в таком случае присутствуют элементы управления кардридером, то они выполняются платой для выбранного по адресу кардридера. Ответ на переданные команды модуль возвращает также в виде строки текста, в которой присутствует результат выполнения последней операции или же код выполнения, который сигнализирует или об успешном выполнении кода или о присутствии ошибки.

1.2 Анализ программного обеспечения для осуществления персонализации средствами эмбоссеров MATICA

В предыдущем подразделе были рассмотрены все теоретические основы работы с эмбоссерами и в частности эмбоссером MATICA и его кардридерным модулем, которые необходимы для понимания контекста работы. В данном же подразделе будут рассмотрены элементы, находящиеся по другую сторону процесса эмбоширования, а именно все программное обеспечение, необходимое для управления эмбоссером и осуществления персонализации пластиковых банковских карт. Так как в процессе персонализации с

использованием эмбоссеров МАТІСА используется не одно единственное ПО, данный раздел разделен на несколько подразделов для каждого из использованных в ходе работы программных средств.

1.2.1 Платформа FloraWare

Программная платформа Floraware является основой, на которой полностью разработан программный продукт TranzWare CardFactory, предназначенный для управления процессом персонализации банковских карт. Сам продукт CardFactory будет рассмотрен подробнее в следующем пункте. В данном же пункте будет подробно рассмотрена структура платформы FloraWare, а также все средства разработки, которые она предлагает для осуществления сетевого взаимодействия с другими устройствами, что необходимо для управления кардридерным модулем эмбоссера как это была описано в предыдущем разделе, и для разработки интерфейсов, что пригодится при доработке модуля настроек.

Не смотря на то, что FloraWare была обозначена как платформа, по своей сути она представляет целую экосистему для разработки и исполнения приложений. Вместе с ней поставляются все необходимые разработанные для нее и на ее основе средства для разработки, отладки и тестирования приложений. FloraWare является уникальным в своем роде программным продуктом, он фактически является отдельной подсистемой для ОС Windows, полностью поддерживаемая компанией Компас плюс.

Рассмотрим подробнее причины возникновения такой платформы как FloraWare. Изначально платформа разрабатывалась с расчетом на то, чтобы упростить процесс разработки для программистов компании и упростить внедрение разработанных решений в клиентские системы. В связи с этим необходимо было разработать такое решение, которое совмещало бы оба этих аспекта. Это значительно сократило бы трудозатраты на разработку и внедрение, что позволило бы в свою очередь сократить количество сотрудников до необходимого минимума.

Со стороны разработки было принято решение частично отказаться от текстового представления программы в сторону объектного подхода к

программированию приложений. Попытки создания полноценной объектной машины принимались неоднократно, но все они ограничивались полумерами. Это можно объяснить несколькими причинами. Технология, меняющая что-то кардинально, требует отбрасывания существующих программных компонентов, созданных по другим технологиям. Поэтому она обязана предложить весь спектр инструментальных средств, либо как запасной вариант, предоставить инструменты поддержки других технологий, и то и другое требует больших усилий. С другой стороны ОО языки программирования создавались с учетом универсальности объектов и полного контроля над ними со стороны пользовательской программы. Даже если технология предполагает наличие некой виртуальной машины, назвать ее объектной можно только с большой натяжкой, на откуп ей в лучшем случае отдается сборка мусора. Тогда как наибольшие преимущества объектной машины сказываются при максимальном ее контроле над объектами. Попытки заменить текстовое представление алгоритма, в большинстве случаев, сводятся к генерации текста части программы на основе ее графического представления с последующей трансляцией, интересным же является полное исключение теста программы. Избавиться от перечисленных недостатков можно, отказавшись от архитектуры машины с потоком команд и линейной памятью, и перейдя к архитектуре с древовидной объектной памятью, с управлением объектами собственно объектной машиной [2,3].

На примере технологии программирования FloraWare сделана попытка реализации полноценной объектной машины как основы реализации объектного метода. С практической точки зрения, целью создания FloraWare было построение простой и удобной объектно-ориентированной среды разработки и исполнения приложений для больших программных систем со средствами коллективной разработки и удаленной отладки приложений.

Объектная машина - основа реализации объектного метода, должна иметь инструменты поддержки неотъемлемых составляющих объекта: *состояния, поведения и идентичности*, а также инструменты реализации отношений между объектами: *связей и агрегации*. В объектной машине

FloraWare в качестве форматов данных выступают объекты, а алгоритм реализуется передачей сообщений между объектами и их реакцией на эти сообщения. Архитектурно объектная машина *FloraWare* может быть представлена в виде нескольких уровней.

На самом нижнем уровне реализован аппарат работы со свойствами объектов, которые относятся к четырем базовым типам: целое, действительное, строка и указатель на объект, эти свойства могут отличаться набором индексов базовых типов. Свойства объекта это инструмент поддержки его *состояния*.

Наличие объектов-контейнеров, способных содержать внутри себя другие объекты, позволяет строить сколь угодно сложные объекты. Обязательный объект - *корневой контейнер*, в который по мере необходимости добавляются другие объекты, реализует память объектной машины представляющую собой дерево, в котором находятся все объекты, существующие в данный момент времени. Добавление всем объектам выделенного строкового свойства *Имя* позволяет *идентифицировать* объекты и осуществлять навигацию по дереву объектов. Составное свойство *Ссылка на объект*, состоящее из относительного пути до объекта и указателя на объект, дает возможность организовывать сколь угодно сложные связи между объектами. Таким образом, древовидная структура данных является основой реализации отношения *агрегация* между объектами, а с помощью ссылок на объекты устанавливаются отношения типа *связь*.

Второй уровень объектной машины обеспечивает работу объектов встроенных базовых классов, которые освобождают пользователя от проектирования самих этих классов, в его обязанности входит только определение использования их экземпляров. Встроенные классы реализованы в ядре *FloraWare* как обычные классы, созданные с использованием объектно-ориентированного языка программирования, со своими свойствами и методами.

Конечно, *FloraWare* не полностью отказалась от текстового представления программ в угоду объектному подходу. Внутри *FloraWare*

присутствуют специальные объекты-функции и объекты-задачи. Они могут содержать в себе код на встроенном языке программирования F++. Он имеет С-подобный синтаксис, но немного другой набор встроенных типов и функций в отличие от того же языка программирования С. После написания кода, его можно скомпилировать внутрь объекта и при успешной компиляции впоследствии использовать эту функцию или задачу внутри других объектов. Между двумя представленными типами объектов, объектами-функциями и объектами-задачами, присутствует небольшая разница и заключается она в методах запуска объектов. Объекты-функции выполняются в пределах других программ как обычные функции в других языках программирования. Объекты-задачи же запускаются встроенным методом `strob`, который выполняет программный код объекта в новом потоке. Таким образом, объекты-задачи представляют из себя метод реализации многопоточности при разработке приложений в экосистеме FloraWare.

Кроме ускорения и упрощения процесса разработки, одним из необходимых требований было упрощение интеграции готовых программных решений в пользовательские системы. Такая проблема была решена путем выстраивания вокруг объектной машины, описанной выше, полноценную подсистему для ОС Windows. По итогам, все необходимые зависимости и библиотеки были скомпилированы в полноценный программный продукт FloraWare, выполнение которого всецело происходит посредством объектной машины. При выполнении любого приложения в среде FloraWare его разработанное объектное дерево из специального файла-объектного хранилища загружается в главное дерево подсистемы. После подгрузки дерево приложения считается инициализированным. Такое дерево активно анализируется и выполняется объектной машиной, до тех пор пока оно будет присутствовать внутри главного дерева подсистемы, когда же приложение будет закрыто, все его инициализированное дерево будет стерто из главного и выполнение его, соответственно, прекратится. Объектная машина содержит внутри себя всю информацию об объектах FloraWare, необходимую для выполнения приложений, поэтому при интеграции

решений, разработанных на базе платформы, возможно обойтись только установкой последней доступной версии FloraWare и загрузкой необходимого программного продукта.

Кроме всего вышеперечисленного, так как платформа всецело разрабатывается компанией Компас плюс, сама компания может постоянно дорабатывать платформу, перерабатывая имеющиеся объекты, содержащиеся в объектной машине, или же создавая абсолютно новые. Таким образом компания может эффективно подстраивать платформу под текущие цели и задачи.

Таким образом, платформа FloraWare является одним из основополагающих инструментов, который позволяет разработчикам компании Компас плюс быстро разрабатывать приложения, эффективно производить их отладку и поиск ошибок и легко интегрировать решения на клиентских системах. Не смотря на то, что помимо представленной платформы, компания параллельно разрабатывает еще несколько схожих по функциональности продуктов, FloraWare не теряет своей актуальности как внутри компании, так и среди клиентов. В особенности, учитывая что на основе данной платформы разработан один из крупнейших программных продуктов компании – TranzWare CardFactory.

1.2.2 Программный продукт TranzWare CardFactory

Программный продукт TranzWare CardFactory полностью разработан на платформе FloraWare. Данный продукт направлен на предоставление клиентам компании средства для настройки этапов обработки банковских карт, хранения этих настроек и непосредственно осуществления такой обработки. Изначально продукт разрабатывался дабы объединить все этапы обработки банковских карт в одном программном решении и предоставить удобный инструмент, с помощью которого можно полностью осуществлять и контролировать весь процесс обработки банковских карт. В данном пункте будет рассмотрена общая структура программного комплекса CardFactory, а также методы осуществления настройки и обработки банковских карт его средствами. Кроме того будут вкратце описаны основные этапы обработки

карты, с заострением внимания на том этапе, во время которого необходимо выполнять взаимодействие с кардридерным модулем эмбоссера.

Как уже упоминалось выше, программный продукт TranzWare CardFactory направлен на предоставление удобных и понятных средств для настройки этапов обработки банковских карт, хранения этих настроек и выполнения всех настроенных этапов. Для этого разработчики придерживались модульной архитектуры при построении продукта. В CardFactory присутствует главное вызывающее приложение, через которое применяются все модули, и непосредственно сам набор всевозможных модулей, предназначенных как для обработки карт, так и для выполнения других утилитарных функций внутри продукта. Внутри вызывающего приложения возможно осуществлять настройку этапов будущей обработки пластиковой банковской карты, это выполняется путем создания специального «карточного продукта». При создании любой карточный продукт внутри CardFactory является абсолютно пустым. Пользователь наполняет его, добавляя в его внутренний список модули, которые в будущем будут использоваться для обработки карты. Каждому модулю внутри карточного продукта соответствуют специальные модули настроек и модули обработки в среде CardFactory.

Модули настроек используются во время настройки этапов обработки и позволяют выставить все необходимые параметры, которые понадобятся в соответствующем модуле обработки. К примеру, возьмем один из модулей настройки – File Loader. Его задачей на этапе обработки является подгрузка данных обрабатываемых карт из одного или нескольких файлов. Внутри данного модуля настроек возможно выставить параметры для таких вещей, как: расположение файлов с данными, способ отбора файлов, шаблон файлов для выбора, и т.д. После выставления всех необходимых параметров, модули сохраняют настройки в специальных файлах или же в базе данных. Каждый модуль осуществляет сохранение параметров по своим внутренним правилам, поэтому останавливаться на анализе сохранения настроек модулей не будем. Всего таких модулей внутри CardFactory имеется 24, из них для полноценной

обработки карты с помощью эмбоссера MATICA и его кардридерного модуля необходимо использовать модули:

- File Loader – для загрузки данных обрабатываемых карт из файлов;
- PIN Generator & Printer – для генерации PIN карты;
- Attributes filler – для заполнения атрибутов карты, таких как сгенерированный PIN, пользовательские поля карты, Service code, MBR, PVKI и т.д. Также может использоваться для генерации PAN карты;
- CVV Calculator – для расчета значения, используемого для подтверждения валидности данных карты (CVV/CVC – Card Verification Value/Card Verification Code) на основании специальных ключей CVK;
- PVV Calculator – для расчета значения, используемого для подтверждения PIN карты (PVV – Pin Verification Value) на основании специальных ключей PVK;
- EMV Preparator – для подготовки данных, записываемых на чип карты. Обычно это специальные ключи, параметры карты, флаги, отвечающие за определение функциональности карты, код валюты и т.д.;
- ODBC Writer – для записи настроенных данных в определенную базу данных;
- Chip Writer Driver – для записи данных на чип карты и настройки процесса;

С модулями обработки же пользователь никак не может взаимодействовать. Они последовательно запускаются, когда пользователь нажимает кнопку выполнения обработки в модуле Processing, при условии, что в начале обработки он выбрал в соответствующем поле модуля необходимый настроенный карточный продукт. Последовательность запуска модулей определяется их положением в списке модулей карточного продукта. Они запускаются вызывающим приложением, начиная с первого в списке. После запуска каждый модуль подгружает настройки из необходимых для него расположений и запускает главную функцию. Такой функцией в каждом модуле является функция Start. Обычно она подгружает все основные компоненты, возможно доподгружает настройки и выполняет необходимые действия модуля. После завершения функции Start, вызывающим

приложением инициализируется следующий по списку модуль и цикл повторяется до тех пор, пока не будет завершена обработка последнего модуля.

Некоторые из модулей обработки имеют достаточно обширный функционал, поэтому они тоже в свою очередь разбиты разработчиками на несколько обособленных модулей, которые при надобности подгружаются вызывающим модулем обработки. Такие «подмодули» будут называться драйверами в контексте данной работы. Обширный список драйверов присутствует у модулей Embosser Driver и Chip Writer Driver, которые отвечают за эмбоссирование пластиковой карты с использованием одного из поддерживаемых в продукте эмбоссеров и запись информации на чип карты с использованием одного из поддерживаемых внутри продукта кардридеров. Так как существует немало эмбоссеров, кардридеров, и видов карт которые используют клиенты компании, задачей разработчиков является осуществление поддержки работы CardFactory со всеми из них. По этой причине к обоим модулям в проекте разработано множество драйверов для работы с различными картами и аппаратным обеспечением.

Подробнее рассмотрим один из необходимых для полноценного выпуска карты модулей – Chip Writer Driver. Для обработки карты этот модуль обязательно подгружает два драйвера: драйвер записи карты и драйвер кардридера. В драйвере записи карты находится весь алгоритм записи определенного типа карты, который находится внутри функции Personalization. Она подгружает всю информацию, необходимую для записи чипа, и, используя функции кардридерного драйвера, осуществляет запись информации на пластиковую карту, подключенную к кардридеру. Драйвер кардридера же в свою очередь всего лишь предоставляет программный интерфейс взаимодействия с аппаратными средствами подключенного к ЭВМ кардридера. Внутри функции Personalization драйвера записи карты используются функции кардридерного драйвера для записи данных на карту через подключенный кардридер, при условии, что в него изначально вставлена карта, иначе же драйвер прервется и будет ожидать карту. В любом драйвере

кардридера присутствует несколько функций, которые используются функцией драйвера записи карты для подключения к кардридеру, настройки и т.д. Такие функции являются общими для всех таких драйверов и все они будут подробнее рассматриваться в следующем разделе. Основной целью данной работы является разработка еще одного кардридерного драйвера, который будет использоваться для взаимодействия с кардридерным модулем эмбоссера MATICA.

Таким образом, программный продукт TranzWare CardFactory является необходимым инструментом при осуществлении обработки пластиковых банковских карт. Он облегчает процессы настройки и обработки, визуально отображает каждый настраиваемый параметр, что дает возможности к внесению более детальных и точных настроек, а также упрощенному их отслеживанию. Он является незаменимым для клиентов компании Компас плюс, так как подобные по масштабам и функциональности решения просто напросто отсутствуют на рынке.

1.2.3 Утилита Maticard PRO

К сожалению, осуществлять взаимодействие со всеми модулями эмбоссера MATICA невозможно без специализированного ПО, а конкретнее утилиты Maticard PRO. Она полностью разработана компанией MATICA в дополнение к своему эмбоссеру и поставляется клиентам вместе с ним. Это программное обеспечение служит для взаимодействия с эмбоссером MATICA и настройки этапов обработки карты на нем. По большому счету, оно очень похоже на продукт CardFactory, но сильно урезанный и осуществляющий работу только с эмбоссерами компании MATICA. Внутри него так же, как и в CardFactory возможно настраивать процесс персонализации, путем добавления модулей в специально создаваемые разделы настроек. Для тестирования кардридерного модуля пригодится только модуль записи чипа.

Так как данное ПО не представляет большой значимости в рамках данной работы, оно будет рассмотрено поверхностно. Порядок запуска обработки карты через Maticard PRO и TranzWare CardFactory с использованием эмбоссера MATICA выглядит следующим образом. Сначала

запускается CardFactory, внутри которого запускается обработка карточного продукта. На одном из этапов обработки CardFactory записывает данные для эмbossирования карты в специальную базу данных, содержащую таблицу данных карт. После этого запускается ПО Maticard PRO, в котором также запускается обработка карты. Maticard подгружает из базы все необходимые данные и производит последовательное эмbossирование всех карт, содержащихся в базе. Кроме того, Maticard произведет остановку обработки на этапе записи чипа для передачи управления процессом в CardFactory. Видите ли, внутри утилиты Maticard PRO возможно передать управление одним из этапов персонализации функциям внешней библиотеки. Разработчиками компании Компас плюс была разработана такая библиотека DLL. Ее функции отправляют CardFactory сигнал о том, что пришло время запуска этапа записи чипа. В этот момент в CardFactory запускается модуль Chip Writer Driver. Он записывает данные на пластиковую карту и подает сигнал обратно о завершении процесса. По принятии сигнала Maticard продолжает свое выполнение.

В процессе описания процесса запуска было упущено множество деталей. Подробнее полный процесс обработки карты будет рассматриваться в разделе анализа текущих решений. Конечно, идеальным решением было бы осуществление полноценного управления эмbossером MATICA через продукт TranzWare CardFactory, в таком случае не пришлось бы прибегать к использованию сторонних решений. К сожалению такой исход не представляется возможным. По итогам консультации с сотрудниками итальянской компании MATICA по данному вопросу, было определено, что никакой документации компания предоставлять для осуществления таких целей не будет, как и разглашать способы прямого взаимодействия с эмbossером. Компанией были предоставлены только ПО Maticard PRO, инструкция по работе с ним, документация и полное описание способов взаимодействия с кардридерным модулем и примеры внешних библиотек для передачи управления. В такой ситуации не остается ничего другого, кроме как воспользоваться поставляемым компанией MATICA программным

обеспечением, даже если такое использование в рамках разработки будет сведено к абсолютному минимуму.

1.3 Анализ текущих решений

На сегодняшний день существуют решения для полноценной персонализации пластиковой карты средствами эмбоссеров MATICA с помощью TranzWare CardFactory, но ни одно из этих решений не взаимодействует с кардридерным модулем эмбоссера, отчего процесс выпуска получается медленным и неэффективным. В рамках данного подраздела будут рассмотрены текущие решения по персонализации карт средствами эмбоссеров MATICA, а также схему работы таких решений. Кроме того в данном разделе будет проанализировано объектное дерево других кардридерных драйверов для выяснения общих компонентов, которые должны присутствовать внутри любого кардридерного драйвера внутри CardFactory для его правильной работы. На основании этих данных будет составлен шаблон объектного дерева кардридерного драйвера, который позже будет использоваться как основа для проектирования и разработки драйвера.

1.3.1 Анализ кардридерных драйверов продукта TranzWare CardFactory

Всего на текущий момент внутри продукта TranzWare CardFactory присутствует 25 кардридерных драйверов, каждый из которых позволяет осуществлять взаимодействие со своим типом кардридера. Конечно, каждый кардридер имеет свою собственную, уникальную, структуру, со всеми необходимыми функциями, объектами настроек и т.д., разработанным специально под кардридер. Не смотря на это, каждый кардридерный драйвер имеет общие с другими элементы, которые необходимы для работы драйверов записи карты. Как было упомянуто в прошлом подразделе, все драйвера записи карты используют функции кардридерных драйверов для взаимодействия с картами через кардридеры, поэтому наличие стандартизированного интерфейса взаимодействия между двумя представленными типами драйверов является обязательным условием для безошибочной обработки карты. Такой интерфейс взаимодействия,

естественно, присутствует между драйверами, иначе средствами CardFactory невозможно было бы без наличия программных ошибок осуществить обработку карт. В данном пункте будет описан такой интерфейс взаимодействия и на основании этого будет выявлен «шаблонный» кардридерный драйвер, структура которого впоследствии будет использоваться на этапах проектирования и разработки драйвера. Стоит упомянуть, что для такого интерфейса взаимодействия между драйверами нету никакой документации в базах компании Компас плюс, поэтому все необходимые элементы интерфейса взаимодействия были выявлены путем анализа структуры существующих драйверов и функций кардридерных драйверов.

Анализируя имеющиеся внутри CardFactory кардридерные драйвера, а также драйверы записи карты, можно прийти к выводу, что для обеспечения взаимодействия между двумя типами драйверов, в кардридерном драйвере обязательно должен присутствовать определенный набор функций, каждая из которых отвечает за определенный этап работы с кардридером. Ниже представлен список таких функций с кратким описанием их назначения:

- EjectCard – реализует процесс закрытия соединения с кардридером.
- Open – используется для подключения к кардридеру и его предварительной настройки
- Restart – перезапускает процесс работы с кардридером, заново передавая ему необходимые настройки и инициализируя подключение.
- Transfer – отправляет данные на карту через кардридер и возвращает ответ, представленный в виде целого числа.
- WaitCard – если карта не была обнаружена в кардридере после подключения и передачи настроек, подключение карты будет ожидаться данной функцией.

Конечно, в каждом кардридерном драйвере реализация данного интерфейса значительно разнится, так как каждый отдельный кардридер реализует собственные способы взаимодействия с ЭВМ. К тому же, это не имеет большого значения, так как изначальной целью было выявление

функций интерфейса взаимодействия драйверов. По большому счету, выявленный список функций и будет представлять шаблонный кардридерный драйвер, так как другие общие объекты необходимые для взаимодействия с драйверами записи карты, у них отсутствуют. Остальные же объекты внутри драйверов используются для индивидуальной реализации взаимодействия с кардридерами. В них присутствуют различные объекты для хранения настроек, объекты подключений к базам данных или другим устройствам по сети, дополнительные библиотеки, функции и т.д.

По итогу, цель данного пункта была достигнута. Были проанализированы структуры кардридерных драйверов в совокупности с кодами функций драйверов записи карты. В результате были выявлены общие элементы кардридерных драйверов, которые необходимы для их безошибочного взаимодействия с драйверами записи карт. Полученный список функций будет использоваться на этапах проектирования и разработки, как основа для будущего драйвера.

1.3.2 Анализ актуального процесса персонализации карт с использованием эмбоссеров MATICA

На текущий момент присутствует возможность персонализировать банковскую пластиковую карту средствами TranzWare CardFactory с использованием эмбоссеров MATICA. К сожалению, средства продукта CardFactory не позволяют осуществлять прямое управление процессом. Все управление осуществляется опосредованно, путем передачи необходимых для персонализации данных, которые впоследствии используются ПО Maticard PRO для исполнения процесса через подключенный эмбоссер. Конечно, идеальной ситуацией было бы полное управление процессом средствами CardFactory, но, к сожалению, итальянская компания MATICA не предоставила средств и информации для поддержки такого функционала внутри продукта. В данном пункте будет рассмотрен актуальный на сегодняшний день процесс персонализации пластиковой банковской карты средствами TranzWare CardFactory и Maticard PRO с использованием эмбоссера MATICA.

Процесс, естественно, начинается с инициализации эмбоссера. На него подается питание и внутренние системы эмбоссера автоматически проверяются и калибруются. После успешной инициализации эмбоссера, на управляющей ЭВМ пользователем запускается ПО CardFactory и Maticard. Конечно, следующим этапом является создание и настройка карточного продукта для обработки внутри CardFactory, но условимся, что необходимый карточный продукт был заблаговременно создан и настроен. После, запускается обработка карточного продукта внутри CardFactory. Процесс обработки приостанавливается на моменте запуска модуля обработки Chip Writer Driver. Изначально, в этом модуле запуск обработки был настроен по получению сообщения из специального xml-файла обмена, поэтому он будет ожидать такого сообщения. После этого, пустые пластиковые карты вставляются в эмбоссера и внутри Maticard PRO также запускается изначально настроенный процесс обработки карты. В нем были настроены все необходимые этапы эмбоширования карты, а также этап записи чипа. Стоит упомянуть, что внутри этапа записи карты был настроен только специальный параметр, позволяющий передать управление процессом функциям внешней библиотеки DLL. Такие функции стандартизированы компанией MATICA и процесс разработки внешних библиотек для передачи управления описан в документации, предоставленной компанией. Библиотека DLL для передачи управления процессом записи чипа была разработана сотрудниками компании Компас плюс. Функции библиотеки открывают xml-файл и записывают в него специально заготовленное сообщение. После завершения процесса эмбоширования, когда внутри Maticard запускается процесс обработки чипа, подгружается внешняя библиотека и передается xml-сообщение в файл обмена. Как только оно записывается в файл, CardFactory регистрирует это и модуль обработки Chip Writer Driver продолжает свою работу. В этот момент необходимо достать карты из эмбоссера и переместить их в отдельный кардридер. После этого CardFactory средствами Chip Writer Driver-а последовательно запишет все карты через подключенный кардридер. По завершению этого процесса, Chip Writer Driver отправит в файл обмена

сообщение о завершении, которое подхватит ожидающая функция библиотеки. После, она завершит свое выполнение, вследствие чего завершит выполнение обработки и Maticard PRO, так как запись чипа будет последним этапом.

Как можно заметить, описанный выше процесс не обладает непрерывностью. После эмбоссирования необходимо прервать процесс персонализации для перемещения карт в отдельный кардридер для осуществления записи. Это существенно замедляет процесс персонализации и создает поле для возникновения ошибок операторами эмбоссера, так как они, например, могут случайно вставить карты в кардридер в неправильном порядке, отчего эмбоссированные данные и данные на чипе карты не будут совпадать и придется полностью перезапускать процесс персонализации карты. Если же процесс будет происходить непрерывно и всецело внутри эмбоссера MATICA, возможно будет избежать ошибок, связанных с человеческим фактором, а также значительно ускорить процесс персонализации карт с помощью эмбоссеров MATICA. Разработка и внедрение драйвера для кардридерного модуля эмбоссера MATICA позволит упростить этап переноса банковских карт в отдельный кардридер, карты сразу же будут подаваться эмбоссером в необходимый кардридерный модуль, отчего процесс персонализации с помощью эмбоссеров MATICA средствами TranzWare CardFactory станет непрерывным.

1.4 Постановка цели и задач

Разработка кардридерного драйвера для модуля обработки Chip Writer Driver представляет из себя долгий и трудоемкий процесс, в ходе которого необходимо изучение всех тонкостей процесса взаимодействия между драйвером и кардридерным модулем, структуры кардридерных драйверов внутри CardFactory и т.д. Кроме обеспечения взаимодействия, кардридер должен выполнять множество других побочных задач. Естественно, что для полноценной и безошибочной реализации драйвера кардридерного модуля эмбоссера MATICA, необходимо четко установить цель, которой необходимо добиться для полноценной разработки драйвера, и задачи, которые

необходимо решить для полного выполнения поставленных целей. Без определения таких ключевых элементов невозможно адекватно начинать любые процессы, связанные с разработкой драйвера. В связи с этим, на основании материала первого раздела возможно составить полные списки задач работы, которые необходимо выполнить для осуществления поставленной во введении цели автоматизации записи информации на чип пластиковой банковской карты средствами TranzWare CardFactory с использованием эмбоссеров компании MATICA. Ниже представлен актуальный список задач, которые необходимо выполнить для исполнения поставленной цели:

- вывести функциональные требования, предъявляемые к драйверу;
- спроектировать функциональную схему будущего драйвера;
- описать инструментальные средства, которые будут использоваться в процессе разработки и отладки драйвера;
- спроектировать модель будущих дополнений к пользовательскому интерфейсу модуля настроек Chip Writer Driver;
- описать архитектуру готового проектного решения драйвера;
- описать разработанное объектное дерево драйвера с указанием назначения каждого элемента;
- предоставить примеры использования драйвера в тестовой среде;

По выполнению представленных выше задач, можно будет с уверенностью сказать, что драйвер прямого управления карддридерным модулем был успешно разработан и внедрен в программный продукт TranzWare CardFactory. После этого его можно будет включить в новую версию продукта, которая впоследствии уйдет клиентам. Конечно, вполне возможно, что разработка драйвера не будет на этом закончена и в будущем потребуются различные доработки в соответствии с запросами клиентов. Не стоит забывать, что разработка любого программного продукта представляет из себя замкнутый цикл и, рано или поздно, придется вернуться обратно на этап разработки. Но все же, не смотря на это, не стоит заострять внимание на

возможных будущих доработках драйвера, а следует сфокусироваться на текущих требованиях, выполнение которых является приоритетным.

1.5 Выводы

Как было упомянуто в предыдущем подразделе, любой процесс разработки представляет из себя долгий и трудоемкий процесс. Он состоит из нескольких этапов, каждый из которых необходимо выполнить для осуществления полноценной разработки программного продукта. Можно сказать, что этап анализа и исследования предметной области является самым важным этапом во всем процессе разработки. Без его выполнения, у разработчиков не будет четкого понимания с чего начинать процесс разработки, с каким программным обеспечением придется иметь дело и т.д. В общем и целом, возникнет множество проблем, которые существенно замедлят процесс и приведут к появлению огромного количества ошибок, неэффективных решений и возможно даже к накоплению технического долга, что естественно не поспособствует снижению трудозатрат на разработку.

В рамках данного раздела было проведено доскональное исследование и анализ всех элементов работы, необходимых для понимания ее контекста и средств, которые будут использоваться в процессе разработки драйвера. В целом, можно подвести краткие итоги данной главы в виде списка с небольшим описанием основных элементов, подвергавшихся анализу:

- Эмбоссер MATICA – сложный технический комплекс, состоящий из нескольких модулей, каждый из которых выполняет определенную операцию с банковской пластиковой картой, целью которого является полноценная персонализация банковской карты для выдачи клиенту. Производится и поддерживается итальянской компанией MATICA.
- Кардридерный модуль эмбоссера MATICA – один из модулей эмбоссера MATICA, который отвечает за запись информации на чип карты. Он состоит из трех элементов: блока питания, управляющей платы и кардридеров. Все взаимодействия осуществляется только через управляющую плату. К плате может быть подключено несколько кардридеров для параллельной записи нескольких карт. Взаимодействие с

модулем осуществляется по сети, по протоколу TCP/IP. У модуля имеется базовый IP-адрес – адрес управляющей платы. Адреса каждого отдельного кардриедра вычисляются добавлением номера ридера к младшему байту базового адреса. Команды модулю отправляются строкой текста с кодом на языке TCPScript. Ответ на выполненные команды модуль посылает также в виде строки текста.

- Кардридер – специальное устройство, которое предоставляет аппаратный интерфейс взаимодействия с чипом пластиковой карты. Карта вставляется в него чипом внутрь. Внутри присутствует набор контактов, которые прижимаются к контактной площадке чипа карты и обеспечивают надежное соединение. Кардридеры применяются как для записи информации на чип, так и для считывания данных.
- Платформа FloraWare – экосистема для разработки и исполнения приложений. Предоставляет все необходимые инструменты для разработки и отладки. Реализует практически полный объектный подход к разработке приложений. Все приложение в экосистеме – объекты и связи между ними. Приложения разрабатываются путем построения их объектного дерева. Каждый объект выполняет свою зарезервированную функцию. Все приложения исполняются специальной объектной машиной в пределах экосистемы. При исполнении приложения, его объектное дерево полностью загружается в главное дерево системы FloraWare, где его уже анализирует и выполняет объектная машина.
- Программный продукт TranzWare CardFactory – программная система, используемая для настройки этапов обработки банковских карт, хранения настроек и выполнения процесса обработки. Имеет модульную структуру. Каждая настройка в пределах CardFactory – карточный продукт. Карточный продукт состоит из набора модулей – этапов обработки карты. Каждому модулю соответствуют модуль настройки и модуль обработки. Модули настройки имеют графический интерфейс для настройки параметров обработки. Все параметры могут записываться или в базу данных или в файлы. Модули обработки последовательно запускаются,

когда запускается обработка модуле Processing. Каждый модуль обработки подгружает настройки и выполняет необходимые действия, после этого запускается следующий. Работа всех модулей записывается в специальную трассу.

- Утилита Maticard PRO – программное обеспечение, поставляемое компанией MATICA со своими эмбоссерами. Позволяет настроить этапы обработки пластиковой карты и выполнить такую обработку средствами подключенного к ЭВМ эмбоссера MATICA. Имеет возможность передачи управления этапом обработки функциям внешней библиотеки DLL.

Таким образом, все приведенные в данном разделе результаты анализа послужат прочной основой для выполнения поставленных цели и задач. Учитывая данные анализа, в следующей главе будет выполнено проектирование проектного решения драйвера прямого управления кардридерным модулем эмбоссера MATICA.

2 ПРОЕКТНЫЕ РЕШЕНИЯ ДЛЯ РАЗРАБОТКИ ДРАЙВЕРА ПРЯМОГО УПРАВЛЕНИЯ ЭМБОССЕРОМ MATICA СРЕДСТВАМИ FLORAWARE

2.1 Описание функциональных требований, предъявляемых к разрабатываемому драйверу

В данном подразделе описываются основные задачи, выполняемые драйвером прямого управления эмбоссером MATICA в соответствии с предъявляемыми техническими требованиями. Функциональные требования служат основой для проектирования, разработки, тестирования и документирования драйвера. Они помогают разработчикам четко понимать, какого результата нужно добиться в конечном итоге, и помогают легче планировать дальнейшую разработку и разбивать весь процесс на отдельные части [5,7]. Все функциональные требования будут разбиты на категории, в соответствии с частями функционала драйвера, который они описывают. Конечно, функциональные требования будут разрабатываться с учетом разработки драйвера на платформе FloraWare и с учетом того, что он будет являться частью программного продукта TranzWare CardFactory.

Список функциональных требований, предъявляемых к драйверу прямого управления эмбоссером MATICA:

1) Требования к подключению и инициализации:

- a) Установка соединения: драйвер должен устанавливать соединение с эмбоссером по протоколу TCP/IP. Необходимо, чтобы при этом эмбоссер находился в одной локальной сети с управляющей ЭВМ или же подключен напрямую в один из портов Ethernet управляющей ЭВМ.
- b) Поддержка нескольких соединений: драйвер должен поддерживать подключение только к одной управляющей плате и иметь возможность обращаться к любому кардридеру этой платы в соответствии с настройками.
- c) Передача сообщений: все сообщения, которые планируются для передачи карте должны посылааться управляющей плате по соответствующему для нужного кардридера IP адресу по протоколу

TCP/IP в виде строки символов, в которой будет содержаться команда передачи данных карте, написанная на языке TCPscript.

- d) Обработка ошибок соединения: если посланное драйвером сообщение не получило ответа от эмбоссера или же в ответ получена ошибка, то необходимо прекратить обработку карты и отобразить сообщение об ошибке в логах обработки или всплывающим окном.

2) Требования к управлению картой:

- a) Передача команд: разрабатываемый драйвер должен вызываться драйвером записи карты и передавать карте только те данные, которые поступают ему от этого драйвера.
- b) Обработка сообщений карты: сообщения, полученные от карты должны передаваться вызывающему драйверу записи карты.

3) Требования к конфигурации и настройкам:

- a) Передача настроек: все настройки должны передаваться в драйвер посредством указания их в интерфейсе модуля ChipWriterDriver.
- b) Загрузка настроек: настройки должны загружаться строго из того расположения, в которое они были записаны через модуль настроек.

4) Требования к производительности:

- a) Минимальные задержки: драйвер должен обеспечивать минимальные задержки при обмене данными с эмбоссером, чтобы максимально повысить скорость работы.
- b) Оптимизация обмена данными: драйвер должен использовать эффективные алгоритмы для обмена данными с эмбоссером, минимизируя количество передаваемых байтов.

Определение функциональных требований это необходимый процесс, который поможет в дальнейших этапах планирования, а также на этапе разработки драйвера. Этот подраздел определил полный список функциональных требований, предъявляемых к разрабатываемому драйверу. Данный список будет активно использоваться в дальнейших этапах разработки функциональной схемы драйвера и проектирования интерфейса. В

следующих подразделах будут рассмотрены нефункциональные требования и ограничения, а также функциональная схема драйвера.

2.2 Функциональная схема разрабатываемого драйвера

Этот подраздел представляет функциональные схемы разрабатываемого драйвера, построенные в соответствии с ГОСТ 19.701-90, демонстрирующие взаимодействие между его основными компонентами и внешними сущностями, такими как эмбоссер и вызывающее приложение. Функциональная схема обеспечивает визуальное представление архитектуры драйвера и помогает понять поток данных и логику управления обработки банковской карты в целом. Будут представлены как общая функциональная схема обработки карточного продукта в программном продукте TranzWare CardFactory, так и функциональная схема модуля обработки Chip Writer Driver, вызывающего кардридерный драйвер. Кроме того, будет представлена функциональная схема взаимодействия драйвера записи карты и кардридерного драйвера эмбоссера MATICA.

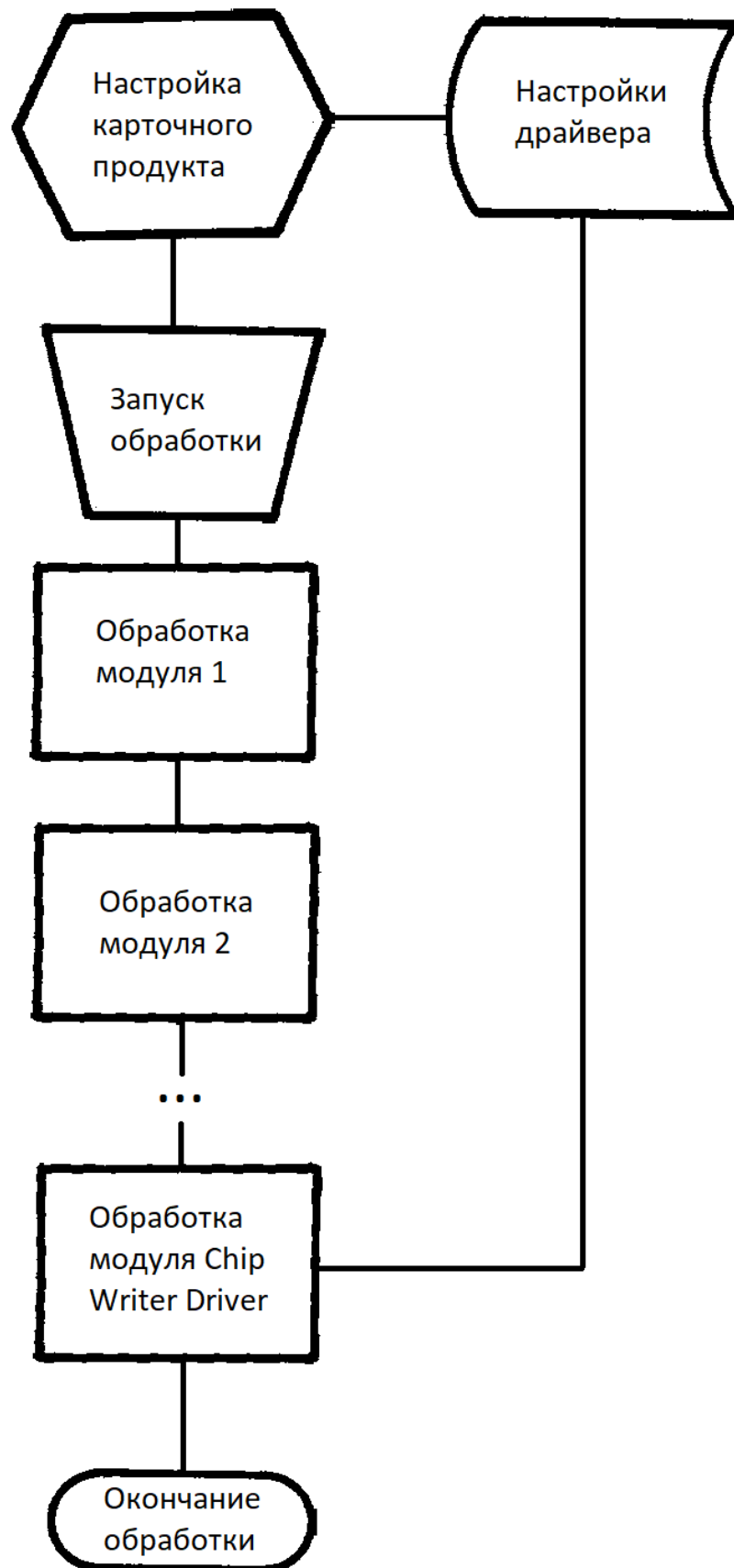


Рисунок 2.1 Общая функциональная схема обработки карточного продукта

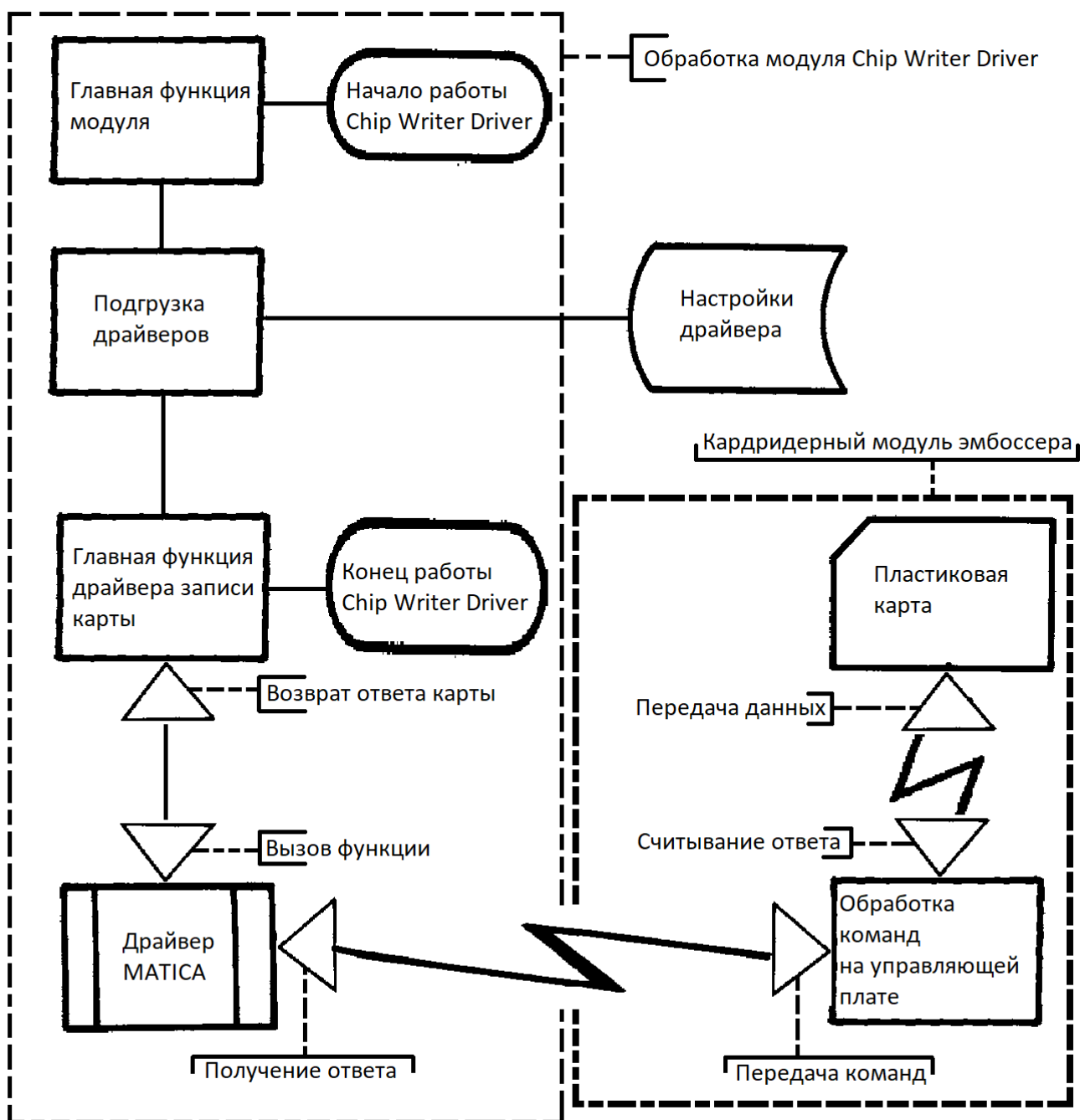


Рисунок 2.2 Функциональная схема работы модуля обработки Chip Writer Driver с использованием драйвера MATICA

Данная схема несет ключевую роль в разработке кардридерного драйвера. Она отражает всю схему работы модуля обработки Chip Writer Driver с использованием будущего кардридерного драйвера MATICA. Полное описание схемы будет представлено в будущих пунктах текущего подраздела.

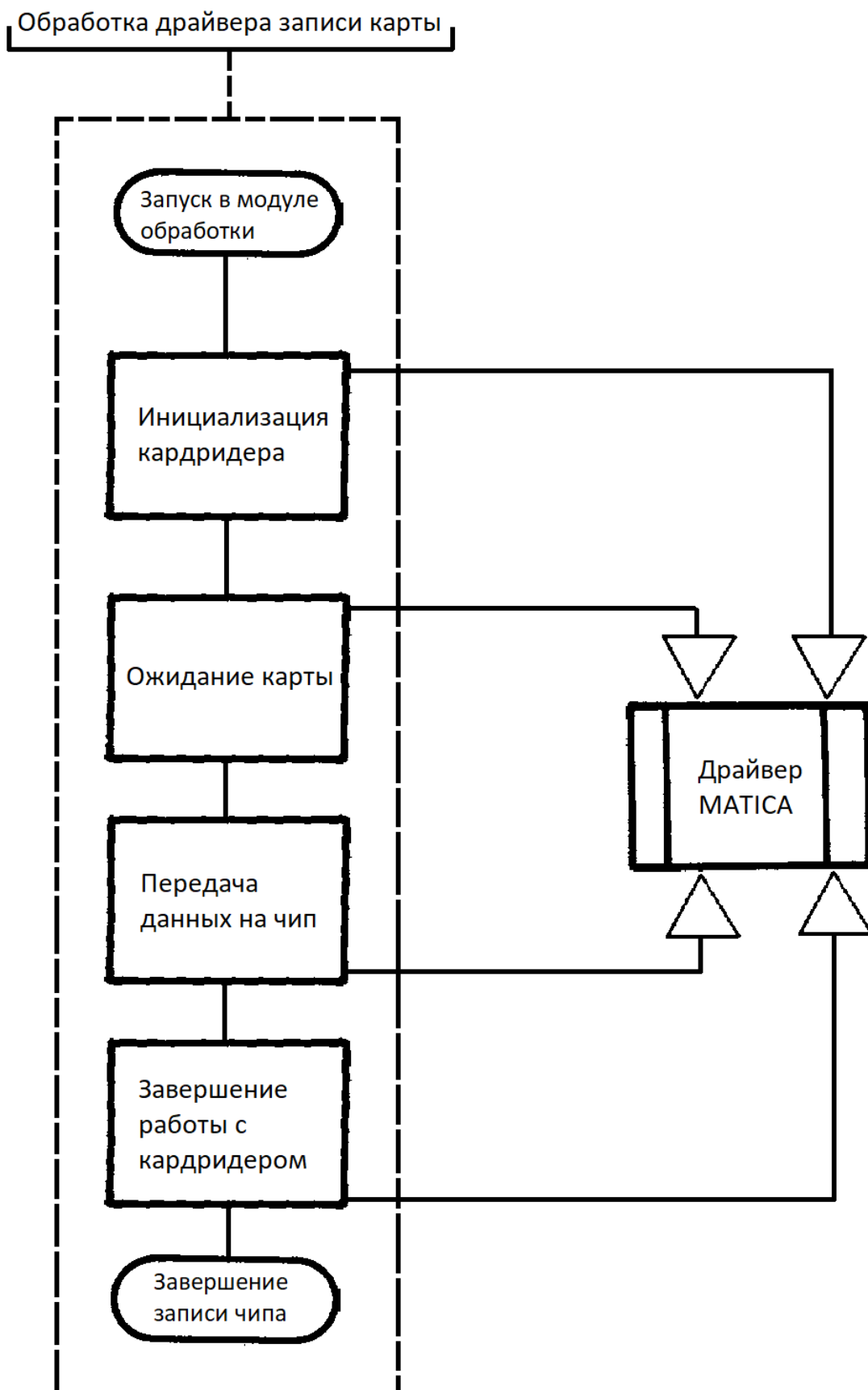


Рисунок 2.3 Общая функциональная схема работы драйверов записи карты и их взаимодействия с кардридерным драйвером

2.2.1 Общее описание функциональных схем

Общую функциональную схему обработки карточного продукта (Рисунок 2.1) можно разделить на два основных этапа: настройку карточного продукта и обработку карты. Схема разделяется таким способом, вследствие того, что процесс настройки карточного процесса достаточно обширный и долгий. Он занимает большую долю усилий и времени, затрачиваемых на весь процесс обработки карты средствами TranzWare CardFactory. На этапе настройки карточного продукта, в интерфейсе модуля настроек Chip Writer Driver, должна присутствовать возможность внесения настроек для проектируемого кардридерного драйвера, которые запишутся в необходимое расположение. Позже, для запуска этапа обработки, пользователем вручную запускается процесс обработки карты. При этом последовательно запускается обработка всех настроенных в карточном продукте модулей. Когда же будет запущен модуль обработки Chip Writer Driver, а это произойдет в самом конце, так как , он должен будет загрузить настройки из расположения и передать их кардридерному драйверу. Такой процесс отражен на схеме потоками данных от блока настройки карточного продукта к блоку настроек драйвера и от блока настроек к блоку обработки модуля Chip Writer Driver.

Далее рассмотрим функциональную схему работы модуля обработки Chip Writer Driver (Рисунок 2.2). Здесь так же можно разделить всю схему на две большие компоненты, взаимодействующие между собой, а именно на процессы, принадлежащие процессу обработки модуля Chip Writer Driver и элементы кардридерного модуля эмбоссера. При инициализации обработки модуля запускается главная функция, которая подгружает драйвер записи карты и кардридерный драйвер, а также загружает настройки кардридерного драйвера и записывает их в него. После запускается главная функция драйвера записи карты, которая должна вызывать функции загруженного кардридерного драйвера для инициализации кардридера, передачи данных на карту и безопасного завершения работы с кардридером. Данные для карты сначала проходят через функцию кардридерного драйвера, после чего передаются в виде команд на управляющую плату кардридерного модуля по

сетевому каналу связи. Как только команды обрабатываются на плате, данные записываются на карту посредством подключенных к чипу кардридерных контактов. После этого поток данных начинает идти в обратном направлении до главной функции драйвера записи карты. Управляющая плата считывает ответ карты на записанные данные и передает его в функцию драйвера MATICA, который ожидает его. Функция драйвера же передает ответ карты в качестве результата своей работы. Так продолжается до тех пор, пока все данные не будут записаны на карту, после чего функция драйвера записи завершает свою работу, отчего завершает обработку и модуль Chip Writer Driver в целом.

В завершение рассмотрим общую функциональную схему работы драйверов записи карты в контексте их взаимодействия с кардридерными драйверами (Рисунок 2.3). На каждом этапе своей работы все драйвера записи карты обращаются к функциям кардридерных драйверов для выполнения тех или иных операций с кардридером. После запуска главной функции драйвера записи карты, он инициализирует кардридер и производит ожидание карты. Это представлено функциями `Open` и `WaitCard` соответственно в кардридерном драйвере. Они непосредственно используются в главной функции для обеспечения выполнения представленных операций. При передаче данных постоянно используется функция `Transfer` кардридерного драйвера, которая отправляет данные на кардридер для записи на карту и получает ответ, который возвращает в качестве результата. При завершении записи карты вызывается функция `EjectCard` кардридерного драйвера, после чего работа главной функции завершается. Как вы можете видеть, функции кардридерного драйвера используются на каждом этапе обработки драйвера записи карты без исключения. Два типа драйверов тесно переплетены между собой и рассматривать их порознь не имеет смысла.

2.2.2 Поток данных и управления

Для лучшего понимания работы описанных функциональных схем представим потоки данных и управления в виде списков для каждой схемы.

Поток данных и управления для общей функциональной схемы обработки карточного продукта:

- 1) Пользователем производится настройка всех модулей карточного продукта, которые будут использоваться для обработки. Все настройки записываются в необходимые расположения.
- 2) Пользователем инициируется обработка настроенного карточного продукта.
- 3) Последовательно запускаются модули обработки, которые подгружают необходимые настройки, а также, если необходимо, результаты работы других модулей.
- 4) Обработка завершается.

Поток данных и управления для функциональной схемы работы модуля обработки Chip Writer Driver:

- 1) Вызывающим приложением TranzWare CardFactory запускается главная функция модуля.
- 2) Происходит подгрузка драйвера записи карты и кардридерного драйвера в зависимости от выставленных ранее настроек, а также загружаются настройки для драйвера записи карты и кардридерного драйвера, которые передаются в них
- 3) Запускается главная функция драйвера записи карты.
- 4) Функция драйвера использует функции кардридерного драйвера для выполнения всех необходимых операций для инициализации кардридера.
- 5) Данные для записи на карту передаются в функцию кардридерного драйвера.
- 6) Кардридерный драйвер составляет набор команд, отправляет его по каналу связи на управляющую плату модуля и переходит в режим ожидания ответа.
- 7) Команды обрабатываются на управляющей плате и данные внутри команд записываются на пластиковую карту через подключенный к кардридеру набор контактов.
- 8) Кардридерный модуль считывает ответ карты

- 9) Ответ передается по каналу связи обратно ожидающей функции кардридерного драйвера
- 10) Функция возвращает полученный ответ в качестве результата работы
- 11) Главная функция драйвера записи карты принимает его и при наличии данных продолжает запись на карту с пункта 5.
- 12) По завершению записи используются функции кардридерного драйвера для завершения работы с кардридером.

Для функциональной схемы работы драйвера записи карты рассмотрение потока данных и управления не требуется, так как сама схема достаточно прямолинейна и призвана только показать насколько работа драйвера сильно зависит от функций сопутствующего ему кардридерного драйвера.

2.2.3 Альтернативные сценарии

В изображаемых функциональных схемах достаточно затруднительно рассмотреть все возможные ситуации и варианты развития событий происходящих процессов. Данный подраздел призван рассмотреть все возможные альтернативные сценарии, которые могут произойти при течении процессов, представленных в рассмотренных функциональных схемах. Все сценарии будут представлены в виде списка с указанием функциональной схемы, в процессах которой он может возникнуть, при каких условиях он возникает и к чему приводит его появление.

Альтернативные сценарии процессов схемы обработки карточного продукта:

- Ошибка настройки: Если один из модулей был настроен неверно или же некоторые необходимые настройки вовсе не были указаны, то будет выброшена ошибка и обработка прервется на том модуле, в котором она произошла
- Ошибка обработки: Во время обработки в одном из модулей произошла ошибка, не зависящая от предыдущего процесса настройки. Выбрасывается ошибка, обработка прекращается.

Альтернативные сценарии процессов схемы работы модуля обработки Chip Writer Driver:

- Ошибка загрузки драйверов: Драйверы были указаны неверно или же отсутствуют файлы, содержащие объектные деревья драйверов для загрузки. Выбрасывается ошибка, обработка прекращается.
- Ошибка соединения: Не удалось установить соединение с кардридерным модулем по сетевому протоколу в отведенное время или же соединение прервалось после подключения в процессе передачи данных. Выбрасывается ошибка, обработка прекращается.
- Ошибка карты: Ответ от карты на записанные данные пришел с кодом ошибки, из чего следует возможное наличие неисправности карты или неверно выставленные настройки. Выбрасывается ошибка, обработка прекращается.
- Ошибка кардридерного модуля: Ответ кардридерного модуля пришел в виде строки текста с описанием ошибки, что сигнализирует об ошибке во время выполнения переданных команд или же неисправности модуля. Выбрасывается ошибка, обработка прекращается.
- Ошибка ожидания: На посланный набор команд от кардридерного модуля за отведенное время не пришел ответ. Выбрасывается ошибка, обработка прекращается.

В целом, это все возможные альтернативные случаи, которые могут произойти при выполнении процессов, представленных в функциональных схемах. Все альтернативные случаи схемы работы драйвера записи карты были рассмотрены как часть процессов схемы работы модуля Chip Writer Driver.

2.3 Описание используемых инструментальных средств разработки

Выбор всех инструментальных средств разработки был продиктован требованиями поставленного технического задания. В данном подразделе будет рассмотрен список таких средств, а также рассмотрены свойства, параметры и функционал данных средств, которые непосредственно влияют на разработку и дальнейшее распространение готового программного продукта.

В качестве ОС для базирования разработки в соответствии с техническими требованиями использовалась система Microsoft Windows. На данной системе разработана вся экосистема FloraWare, так что она является основополагающей при осуществлении любых разработок для FloraWare. Система Windows обладает качествами, которые напрямую повлияют на тестирование и распространение готового драйвера, а именно:

- Широкое распространение: Windows является наиболее распространенной ОС на рынке, что обеспечивает широкую совместимость с различными эмбоссерами и программным обеспечением, которое будет использовать драйвер, в том числе с ПО для работы с эмбоссером MATICA.
- Совместимость с оборудованием: Windows имеет обширную поддержку аппаратного обеспечения, что упрощает интеграцию с различными моделями эмбоссеров.
- Поддержка программной платформы: Windows нативно поддерживает программную платформу FloraWare, на которой производится вся разработка.
- Популярность среди клиентов: подавляющее большинство клиентов CompassPlus, пользующихся программным продуктом CardFactory, используют Windows для взаимодействия с продуктом CardFactory и эмбоссерами.

При возникновении у клиентов необходимости поддержки драйвера на других ОС, присутствует возможность внесения драйвера в версию приложения, разрабатываемую под семейство систем Linux, но в таком случае драйвер придется отдельно вносить и пересобирать на данной системе.

В качестве языка программирования для разработки драйвера использовался язык F++, который является встроенным в программную платформу FloraWare. Данный язык используется для разработки любого программного продукта на базе FloraWare и является единственным языком, выполняющим подобную функцию внутри экосистемы. Весь набор инструментов, присутствующих в языке, построен на основе требований к

инструментарий для разработки продуктов, разрабатываемых на основе FloraWare, таких как CardFactory. Любой язык программирования напрямую влияет на процесс разработки и тестирования продуктов, а также способы распространения и F++ не является исключением. Он обладает всеми необходимыми параметрами для осуществления разработки драйвера, а именно:

- Широкие возможности развития: язык F++ постоянно расширяется, пополняясь новыми встроенными функциями и объектами. Так как язык полностью поддерживается и дорабатывается компанией CompassPlus, то при возникновении необходимости в том или ином функционале присутствует возможность отправки запроса на добавление нового функционала, после чего данный запрос будет рассмотрен и при одобрении внесен в будущую версию FloraWare.
- Минимально необходимый функционал: встроенный в FloraWare язык программирования предоставляет минимально необходимый функционал для разработки поддерживаемых на данной платформе программных продуктов, из-за этого он не является настолько «раздутым», как достаточно популярные на данный момент языки C++, C#, Java, где есть множество вариантов решения одной и той же задачи встроенными средствами языка, а также огромное множество встроенных библиотек и модулей. Язык F++ в свою очередь предлагает более скромный, но достаточный для разработки инструментов, а также он не разбит на огромное множество модулей, все встроенные элементы доступны сразу, без необходимости дополнительного подключения.
- Привычный синтаксис: синтаксис языка F++ разработан как C-подобный, следовательно, разработчики смогут быстрее и без больших усилий приспособиться к данному языку программирования.

При разработке на базе программной платформы FloraWare возможности выбора подходящей среды разработки не так широки, как в системе Windows, других ОС и программных платформах. В данной экосистеме существует единственная доступная встроенная среда разработки

Designer. Среда разработки предоставляет разработчику все необходимые инструменты для облегчения и ускорения процесса разработки. Designer разработан с целью служить ключевым средством разработки экосистемы FloraWare и он обладает всеми необходимыми свойствами для полноценной реализации разрабатываемого драйвера, такими как:

- Полная объектная библиотека платформы: среда разработки Designer предоставляет возможности для просмотра полной библиотеки объектов платформы и добавления элементов этой библиотеки в приложение, отчего разработка становится более быстрой и интуитивной.
- Удобный пользовательский интерфейс: Designer предоставляет удобный и интуитивно понятный пользовательский интерфейс для просмотра и редактирования дерева объектов разрабатываемого программного продукта, а также посредством интерфейсных элементов, он предоставляет возможности взаимодействия с этим деревом, такие как копирование и вставка объектов дерева, добавление и удаление объектов, поиск объектов в дереве по различным фильтрам и т.д.
- Инструмент «Палец»: инструмент «Палец» является одним из наиболее полезных инструментов, встроенных в Designer. Он позволяет указать на любой интерфейсный элемент в любом открытом в среде программной платформы приложении и открыть его инициализированное дерево или же показать элемент, на который указал палец в файле объектного хранилища. Данный инструмент позволяет существенно ускорить разработку, так как упрощает процесс поиска в файлах хранилищ и объектных деревьях нужного графического элемента или модуля, а также позволяет ускорить отладку и тестирования разработанных приложений, так как позволяет открыть инициализированное дерево, в котором можно будет наблюдать текущее состояние объектов и их изменение в реальном времени.

Кроме средств разработки платформа FloraWare закономерно имеет необходимые встроенные средства отладки и тестирования разработанных продуктов, которые комплементируют имеющиеся средства разработки.

Таким средством является приложение Debugger. Не нужно упоминать, что наличие таких средств в платформе позволяет облегчить процесс обнаружения проблем в работе модулей и их исправления, а также предоставляет инструменты для наблюдения потока выполнения программы, для обнаружения неэффективных реализаций и неочевидных ошибок. Debugger обладает рядом свойств, которые делают его незаменимым при осуществлении любых разработок в экосистеме FloraWare, а именно:

- Интеграция с Designer: при условии, что внутри кода какого-либо объекта-функции или объекта-задачи была установлена точка останова с помощью утилиты Designer, а также при условии, что запущена утилита Debugger, во время выполнения кода данного объекта при достижении точки останова Debugger остановит запущенное дерево и предоставит возможность управлять потоком выполнения программы, а также просматривать содержимое локальных переменных и объектного дерева. Таким образом, используя в тандеме утилиты Designer и Debugger, разработчик может добиться высокой производительности своей работы, так как может параллельно пошагово выполнять код в Debugger и просматривать изменения в дереве внутри Designer.
- Минимальный необходимый интерфейс: Debugger является самостоятельной утилитой, отделенной от среды разработки платформы, отчего интерфейс и функционал Debugger-а предоставляет только все необходимое для управления потоком выполнения программы, отлавливания точек останова и просмотра стека памяти программы, что помогает разработчику легче приспособиться к работе с утилитой и ускоряет процесс разработки.

Также, в продукте CardFactory присутствует собственный инструмент отладки и тестирования в виде логов обработки карты, которые записываются каждым модулем и логируют каждое их действие и которые будут использоваться для отслеживания точности информации, передаваемой карте через драйвер, а также для отслеживания ответа, принимаемого драйвером от карты на посылаемые данные.

Кроме инструментов для разработки и тестирования, ни один крупный проект по разработке не обходится без системы управления версиями(VCS – version control system). Она помогает командам разработчиков эффективнее вместе работать над одним проектом. С помощью таких систем любой разработчик в команде сможет отслеживать выполненную работу, а также сможет загружать свои изменения в общий репозиторий, причем современные системы управления версиями предупреждают разработчиков о пересекающихся изменениях, что позволяет избежать перетирания изменений. Для разработки программного продукта TranzWare CardFactory используется клиент Tortoise SVN для сервера VCS Apache Subversion. Функционал и характеристики клиента Tortoise SVN напрямую влияют на процесс разработки продукта, а также на дальнейшее внесение готового решения в рабочую версию CardFactory, конкретнее:.

- Удобный интерфейс: интерфейс Tortoise SVN предлагает очень удобное взаимодействие с репозиториями. Данная система не предоставляет общий универсальный интерфейс, но предоставляет несколько отдельных окон для выполнения операций, таких как клонирование репозитория, откат к ревизии, просмотр списка ревизий. Также хотелось бы отметить, что интерфейс для просмотра списка ревизий выполнен очень удобно и предоставляет всю необходимую информацию о ревизиях, такую как описание ревизии, номер ревизии и список файлов, что позволяет оперативно получать информацию о ревизиях и производить быстрый поиск по списку.
- Проработанная интеграция с ОС Windows: Tortoise SVN очень хорошо интегрируется в ОС Windows, добавляя собственный функционал в контекстное меню папок с репозиториями, позволяя осуществлять взаимодействие с репозиторием через контекстное меню проводника. Также, утилита изменяет пиктограммы папок и файлов, если они находятся под версионным контролем, добавляя к ним небольшое изображение в зависимости от их статуса: соответствует ревизии –

зеленая галочка, файл изменен – красный крест, конфликт с ревизией – желтый треугольник с черным восклицательным знаком.

Набор инструментальных средств разработки является основополагающим фактором для разработки любого программного продукта. Определение такого набора напрямую влияет на скорость разработки, количество допускаемых ошибок и возникаемых багов, масштабируемость проекта, способы распространения проекта и т.д. Инструментальные средства должны быть обязательно определены до начала процесса разработки для того, чтобы четко выбрать самые эффективные для поставленной задачи средства и не останавливать процесс разработки для подбора необходимых инструментов. А иначе возможна такая ситуация, что в процессе разработки проекта будут выявлены недостатки того или иного инструмента и возникнет необходимость перехода на другой. В больших проектах такие переходы могут быть очень трудозатратными, поэтому необходимо по возможности их избегать и четко определять набор инструментов, который будет использоваться на протяжении всей жизни проекта, еще на этапе планирования. Конечно, присутствует возможность оставить уже используемые инструменты, но в таком случае разработчикам придется мириться с их недостатками, что существенно снизит производительность. Представленный в предыдущем разделе инструментарий удовлетворяет все требованиям для осуществления полноценной разработки и тестирования драйвера прямого управления эмбоссером MATICA. Часть представленного инструментария будет использована в следующем разделе для описания процесса проектирования пользовательского интерфейса.

2.4 Проектирование пользовательского интерфейса

В данном подразделе описывается проектирование доработок пользовательского интерфейса (UI) для модуля ChipWriterDriver. UI предназначен для упрощения настройки и управления драйвером, а также для обеспечения удобства взаимодействия с пользователем. В данном случае, мы сосредоточимся на конкретных изменениях, разработанных для

осуществления полноценной настройки драйвера управления эмбоссером, которые планируются к внесению в интерфейс.

2.4.1 Общие принципы проектирования интерфейса

При проектировании доработок к UI для ChipWriterDriver были учтены следующие принципы:

- Простота и понятность: Назначение добавляемых интерфейсных элементов должно быть интуитивно понятно. Взаимодействие с элементами должно быть простым даже для неопытных пользователей.
- Функциональность: Добавляемые интерфейсные элементы должны предоставлять доступ ко всем основным настройкам драйвера, таким как указание базового IP-адреса, порта и режима взаимодействия.
- Обратная связь: UI должен предоставлять пользователю обратную связь в случае несоответствия настроек необходимым шаблонам.
- Соответствие стандартам: UI должен соответствовать общепринятым стандартам проектирования интерфейсов для используемой операционной системы.

Для разработки любого UI необходимо определить принципы, которых должен придерживаться готовый интерфейс. Без их определения велик риск получить негативный отзыв от клиентов, которые получают доработки, или же пропустить какой-либо важный компонент [6].

2.4.2 Планирование интерфейсных доработок

В рамках данной дипломной работы планируется внести изменения в интерфейс модуля ChipWriterDriver для соответствия требованиям к возможности настройки параметров драйвера со стороны пользователя. В данном пункте будут перечислены все планируемые к добавлению в UI интерфейсные элементы с полным описанием всех их составляющих, их примерного расположения и принципов взаимодействия пользователя с этими элементами. Все представленные ниже элементы будут расположены все вместе, недалеко друг от друга, ориентировочно в нижней части модуля, в новом разделе, который будет отмечен рамкой под заголовком «Настройки

Smartware» в русскоязычной версии интерфейса и «Settings via Smartware» - в англоязычной.

Список спроектированных элементов, планируемых к добавлению в интерфейс модуля ChipWriterDriver для осуществления настройки драйвера прямого управления эмбоссером:

1) Поле ввода IP-адреса:

- a) общее описание: элемент представляет из себя текстовое поле для ввода, позволяющее пользователю ввести IP-адрес эмбоссера с клавиатуры или вставить;
- b) расположение: поле расположено в левой части раздела настроек соединения, в нижней части интерфейса модуля;
- c) тип поля: текстовое поле, принимающее только валидные IP-адреса
- d) метка: «Базовый IP-адрес:»;
- e) поведение: поле активно только в том случае, если отмечен флажок опции сетевого подключения (см. ниже), а также выбран необходимый тип эмбоссера. Валидность IP-адреса проверяется при попытке сохранения с помощью заранее заготовленного регулярного выражения при снятии фокуса с поля или нажатии клавиши Enter при активном фокусе на поле.

2) Флажок (checkbox) выбора режима подключения:

- a) общее описание: элемент представляет из себя квадратный флажок, отметка которого меняется при нажатии левой кнопкой мыши;
- b) метка: справа от флажка надпись «Использовать TCP» в русскоязычной версии и «Use TCP» - в англоязычной;
- c) поведение:
 - 1) если флажок отмечен (checked), поле ввода IP-адреса перейдет в активный режим и пользователь может ввести IP-адрес эмбоссера в соответствующее поле;
 - 2) если флажок не отмечен (unchecked), поле ввода IP-адреса становится неактивным;

3) по умолчанию флажок не отмечен, и используется стандартный режим подключения через интерфейс USB.

3) Рамка (frame) настроек подключения:

- a) общее описание: элемент представляет из себя рамку, имеющую тонкие границы, с надписью в левой верхней части рамки;
- b) метка: «настройки Smartware» в русскоязычном интерфейсе и «Smartware settings» - в англоязычной;
- c) все перечисленные выше элементы, планируемые для внесения в интерфейс будут находиться внутри данной рамки;
- d) ширина и высота рамки будут выставлены таким образом, чтобы вмещать все необходимые элементы с учетом отступа от границ в 10 пикселей.

Кроме взаимодействия между собой, представленные выше интерфейсные элементы будут взаимодействовать также и с другими элементами, которые уже присутствуют в интерфейсе модуля ChipWriterDriver. Данное взаимодействие будет проявляться только при выборе типа кардридера в поле выбора контактного интерфейса взаимодействия. Если будет выбран тип ридера «Embossing smartware via ...» то флажок представленный выше станет активным, иначе же он будет не активен и не выствален.

Представленные в данном разделе интерфейсные элементы будут использоваться для проектирования модели будущего интерфейса, которая уже будет служить основой для доработки реального интерфейса модуля.

2.4.3 Визуализация интерфейса

С помощью веб-инструмента Figma была составлена приблизительная упрощенная модель будущего интерфейса модуля настроек Chip Writer Driver [4]. Такая модель поможет в дальнейшем упростить внесение доработок в интерфейс и поможет выявить возможные проблемные моменты, которые необходимо будет учесть до изменения интерфейса.

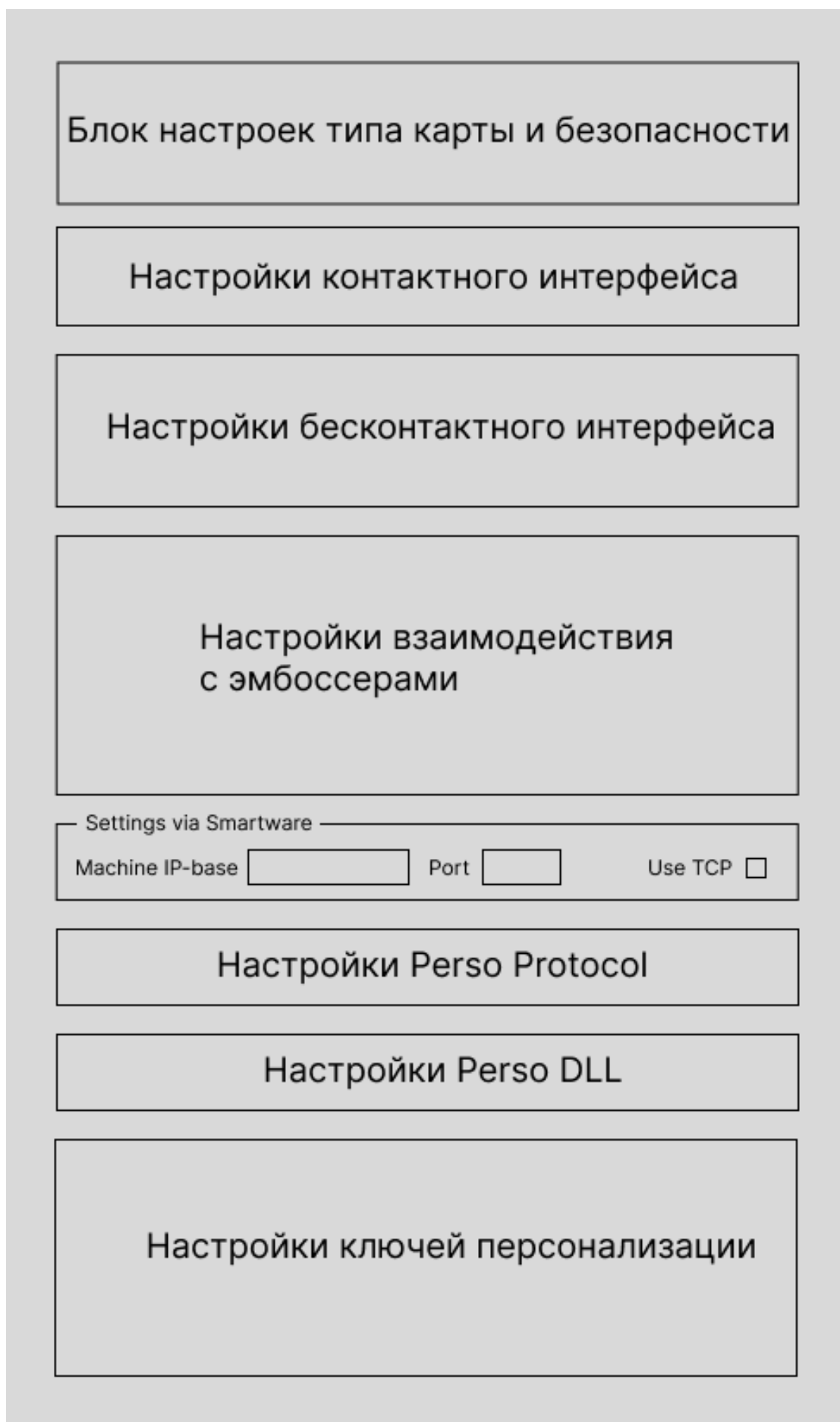


Рисунок 2.4 Упрощенная модель будущего интерфейса Chip Writer Driver

Как можно заметить, все множество блоков настроек в модели на рисунке 2.4 представлено в упрощенном варианте, детально смоделирован только добавляемый в интерфейс блок настроек кардридерного драйвера. Причем стоит отметить, что модель четко передает то, как интерфейсные элементы отрисовываются в платформе FloraWare. При указании заголовка рамки, он всегда будет находится в левой верхней ее части и обрывать собой саму рамку. Поля для ввода базового адреса и порта будут отображаться точно так же. Заголовки полей всегда будут находится непосредственно слева от самого поля, как и заголовок галочки использования сетевого взаимодействия, которая находится в правой части блока. Стоит упомянуть, что галочка была добавлена по рекомендации сотрудников компании для будущего добавления других режимов взаимодействия с кардридерным модулем эмбоссера. Пока что данная галочка будет выставляться по умолчанию при выборе контактного интерфейса как кардридерного модуля эмбоссера.

2.5 Выводы

В данном разделе были определены основные аспекты разработки драйвера прямого управления эмбоссером, начиная от функциональных требований и заканчивая проектированием пользовательского интерфейса. На основе проведенного анализа можно сделать следующие выводы:

- **Функциональные требования:** Были четко сформулированы функциональные требования к драйверу, охватывающие все необходимые операции для управления эмбоссером, включая подключение, инициализацию, управление картой и обработку ошибок. Эти требования послужат основой для реализации драйвера и его тестирования. Особое внимание уделено требованиям к обработке ошибок и безопасности, что обеспечит надежную и безопасную работу системы.
- **Функциональная схема:** Разработанная функциональная схема позволила визуализировать архитектуру драйвера и определить взаимодействие между его основными компонентами. Схема наглядно демонстрирует

поток данных и управления, что упрощает понимание логики работы драйвера и его дальнейшую разработку. Разделение на уровни абстракции (вызывающее приложение, API, ядро драйвера, аппаратный интерфейс, эмбоссер) обеспечивает модульность и гибкость системы.

- Выбор инструментальных средств: Был выбор инструментальных средств разработки, учитывая требования к производительности, совместимости, удобству отладки и наличию документации. Выбор операционной системы Windows, языка программирования F++ и Designer с встроенным компилятором языка обеспечивает доступ к обширной библиотеке объектов и мощным инструментам для разработки драйверов. Использование системы управления версиями Tortoise SVN обеспечивает эффективную совместную работу и отслеживание изменений в коде.
- Проектирование пользовательского интерфейса: Был спроектирован пользовательский интерфейс, ориентированный на простоту, понятность и функциональность. В частности, был спроектирован полный набор интерфейсных элементов, подлежащих внесению в интерфейс модуля Chip Writer Driver и обеспечивающих возможность указания всех доступных для кардридерного драйвера настроек.

Основные итоги главы:

- сформированы полные и детальные функциональные требования, необходимые для дальнейшей разработки драйвера;
- разработана функциональная схема, обеспечивающая наглядное представление архитектуры и логики работы драйвера;
- обоснован выбор оптимальных инструментальных средств разработки, обеспечивающих эффективность и надежность процесса разработки;
- спроектирован пользовательский интерфейс, обеспечивающий удобство настройки и управления драйвером.

Таким образом, данный раздел заложил прочный фундамент для дальнейшей разработки драйвера прямого управления эмбоссером.

3 ОПИСАНИЕ И ПОРЯДОК РАБОТЫ С РАЗРАБОТАННЫМ ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ

Этот раздел посвящен подробному описанию разработанного драйвера прямого управления эмбоссером. В нем будут рассмотрены архитектура драйвера, его компоненты, а также примеры использования для выполнения записи информации на карту. Целью этого раздела является предоставление полного руководства для разработчиков и пользователей, желающих использовать разработанный драйвер в своей работе. Кроме того, целью раздела является полное описание проектного решения драйвера.

3.1 Структура драйвера

Как было отмечено в прошлых разделах, все драйверы для работы с кардридерами имеют одинаковую общую структуру, которой обязательно необходимо придерживаться, чтобы драйверы записи карты могли беспрепятственно использовать его функции. В данном подразделе сначала будет рассмотрена общая архитектура разработанного драйвера, а потом будет подробно рассмотрено разработанное объектное дерево с описанием всех взаимодействий и связей между объектами. Кроме того, в одном из пунктов данного подраздела будет подробно рассмотрен и описан разработанный код основных объектов-функций драйвера, обеспечивающих его взаимодействие с кардридерным модулем. В дополнение к этому будут описаны доработки интерфейса модуля настройки Chip Writer Driver, разработанные для обеспечения возможности передачи настроек в драйвер.

3.1.1 Архитектура драйвера

В рамках разработки программных систем, взаимодействующих с аппаратными ресурсами, драйверы выполняют критическую роль посредника между операционной системой и физическими/виртуальными устройствами. Эффективность, надежность и безопасность драйвера напрямую зависят от его архитектурных решений, что обуславливает необходимость разработки надлежащей по требованиям архитектуры [8]. В данном пункте будет вкратце описана архитектура разработанного драйвера управления кардридерным

модулем с подробным описанием взаимосвязей между архитектурными элементами. Подробные описания методов и свойств используемых объектов будут приведены в следующем подразделе при рассмотрении объектного дерева.

Архитектуру разработанного драйвера можно разделить на 4 основные части в соответствии с возможными этапами работы драйвера:

- 1) Механизм инициализации
- 2) Механизм настройки
- 3) Механизм отключения
- 4) Механизм взаимодействия

Механизм инициализации представлен объектами `port` и `sAddress`, а также объектами-функциями `WaitCard`, `CardConnect` и `Open`. Объект `port` представляет из себя объект вида `NetPort`. Внутри `FloraWare` объекты такого типа позволяют подключаться к другим устройствам по сетевому протоколу TCP/IP и выполнять взаимодействие с ними, путем двустороннего общения с подключенным устройством. Задание IP-адреса подключения, передача и принятие сообщений, а также подключение осуществляются путем использования внутренних свойств и методов объекта. `sAddress` – это обыкновенная переменная типа `Str`. Она представляет из себя строку текста с IP-адресом, данные в которую передаются во время подгрузки драйвера в модуле обработки `Chip Writer Driver`. Объект-функция `Open` же содержит код настройки объекта `port` и инициализации подключения к кардридерному модулю по сети с помощью его методов. После функции `Open` драйвером записи карты обязательно вызывается метод `WaitCard`, который ожидает подключения карты, но, так как кроме непосредственного ожидания необходимо выполнить настройку кардридера, а также инициализацию необходимой переменной для управления картой, для облегчения понимания разработчиками, весь процесс ожидания и настройки вынесен в отдельный объект-функцию `CardConnect`, название которого лучше отражает происходящие процессы. После выполнения представленных процессов,

механизм инициализации завершает свою задачу и управление передается на механизм взаимодействия.

Механизм взаимодействия представлен двумя объектами-функциями Transfer и TCPSend. Драйвером записи карты постоянно вызывается функция Transfer для передачи данных на карту. Внутри этой функции содержится код, собирающий команды для отправки на кардридерный модуль. После сборки команды отправляются на ридер посредством вызова функции TCPSend. Внутри нее происходит проверка подключения к модулю с помощью объекта port. Если подключение отсутствует, то функция заново передает необходимые настройки в объект и выполняет попытку подключения. После этого с помощью метода Send объекта port данные передаются по сетевому каналу на модуль. В завершении, функция ожидает сообщения от модуля и по готовности принимает его методом Recv и возвращает в виде строки текста. Стоит упомянуть, что функция TCPSend вызывается два раза внутри функции Transfer. Первый раз для отправки данных на карту, а второй для считывания ответа карты с чипа. Такое разделение присутствует для безопасности работы с модулем, так как если первая команда вернет ошибку со стороны модуля, то необходимости считывать ответ карты не будет. Функция Transfer же возвращает целое число – полученный от карты код результата выполненного действия.

В плане обработки исключительных ситуаций, в модуле отсутствуют для этого какие-либо средства, так как он всего-лишь предоставляет необходимые функции для взаимодействия. Единственная исключительная ситуация в модуле возникает при вызове функции TCPSend в случае получения строки с исключением от модуля. При этом выбрасывается исключение, которое отлавливается драйвером записи карты и он завершает обработку. Сообщение полученного исключения выводится в трассу и отображается пользователю в специальном окне.

Механизм отключения составлен объектами-функциями EjectCard и CardClose. В данном случае логика работы функций аналогична функциям WaitCard и CardConnect, рассмотренным выше. Функция EjectCard вызывается

драйвером записи карты для завершения процесса записи карты. Так как ее название не отражает действительности, была создана функция CardClose, которая вызывается ей. В этой функции передаются команды закрытия подключения и отключения кардридера. Чтобы обработка драйвера записи карты не прерывалась на этапе закрытия при появлении исключения, оно отлавливается внутри функции EjectCard и просто пишется в трассу. Это сделано для того случая, если после обработки чипа присутствуют модули обработки, ожидающие запуска. Так как этап закрытия и отключения кардридера не так важен в процессе обработки карты, если он будет пропущен по возникновению ошибки это будет не критично и карту можно будет дальше обрабатывать.

Механизм настройки же осуществлен всецело в модулях настройки и обработки Chip Writer Driver. В интерфейс модуля настройки были добавлены элементы как это было описано при проектировании в разделе 2.4. В интерфейсе появилась дополнительная рамка с надписью «Settings via Smartware». Внутри рамки было добавлено два поля для внесения настроек, а именно поле «Machine IP-base» для указания базового IP-адреса управляющей платы и поле «Port» для указания порта подключения к плате. Кроме того, была добавлена галочка «Use TCP» для выбора режима подключения, так как у кардридерного модуля существует несколько способов подключения в зависимости от модели. Пока что эта галочка используется только для сетевого режима работы с кардридерным модулем. Остальные режимы будут позже реализованы разработчиками компании в рамках других задач. Все настройки, записанные внутрь полей, пишутся в специальный .ini файл, из которого потом считываются модулем обработки. При подгрузке разработанного кардридерного драйвера модуль обработки Chip Writer Driver записывает считанные настройки адреса и порта из файла в объект драйвера sAddress, из которого позже драйвер их достает для инициализации подключения. К тому же, в модуль настроек Chip Writer Driver было внесено два вида контактных кардридеров, это «Embossing via Smartware interface» и «Embossing via multiple Smartware interfaces». Первый используется для выбора режима однопоточной

записи, при которой используется один кардридер для записи карт. В таком режиме инициализация происходит так, как это было описано выше. Второй же используется для выбора многопоточного режима записи. Этот режим использует несколько кардридеров для параллельной записи карт. При условии выбора такого режима процесс передачи настроек и инициализации будет немного разниться. При запуске модуля обработки Chip Writer Driver, запускается функция Start, в которой происходит инициализация всех необходимых составляющих и загрузка настроек. При выборе однопоточного режима записи, внутри функции Start происходит загрузка кардридерного драйвера в дерево и в него передаются считанные из файла настройки. В случае выбора многопоточного режима запускается функция StartReaderInterfaces, внутри которой для каждого потока записи создается свой отдельный раздел в дереве. В каждый такой раздел загружается инстанция кардридерного драйвера. В этой же функции вычисляется IP-адрес для каждого кардридера и записывается в необходимый объект в каждой инстанции. В общем, инициализация драйвера для обоих режимов не сильно разнится. Отличие состоит только в необходимости передачи настроек в несколько инстанций драйверов в многопоточном режиме, вместо одного в однопоточном.

3.1.2 Объектное дерево драйвера

Рассмотрение архитектуры драйвера необходимо для общего понимания структуры и внутренних механизмов разработанного решения. Описанная таким образом архитектура позволяет осознать только поверхностно те основополагающие механизмы, которые стоят за работой драйвера. Для получения полной картины разработанного программного решения, необходимостью является описание программного кода решения и, в связи с тем, что продукт разработан на базе платформы FloraWare, описание объектного дерева. В данном пункте будут в полной мере описаны оба представленных компонента для разработанного драйвера кардридерного модуля, а также для модулей, в которых в рамках данной работы производились доработки.

Начнем, естественно, с объектного дерева разработанного драйвера кардридерного модуля. Сначала приведем полный список объектов и дадим краткое описание каждому:

- `void CardClose()` – объект-функция. Используется для завершения работы с кардридером. Выключает кардридер и очищает настройки;
- `void CardConnect(str interfaceType)` – объект-функция. Используется для настройки кардридера и ожидания карты. Кроме того, осуществляет переподключение к кардридерному модулю, если соединение по какой-либо причине было разорвано;
- `void EjectCard()` – объект-функция. Часть интерфейса взаимодействия с драйвером записи карты. Используется для завершения работы с кардридером. В случае с разработанным драйвером вызывает функцию `CardClose`;
- `Str interfaceType` – объект-переменная типа `string`. Представляет из себя строку текста с указанием типа кардридерного интерфейса. Может принимать значения «contact» или «contactless». Используется в некоторых функциях;
- `void Open(RStr address)` – объект-функция. Часть интерфейса взаимодействия с драйвером записи карты. Открывает подключение к кардридерному модулю используя методы объекта `port`. Кроме того, выполняет предварительную настройку объекта, прописывая в него адрес подключения из объекта `sAddress`. В реализации драйвера входной параметр `address` передается в функцию после загрузки драйвера;
- `NetPort port` – объект типа `NetPort`. Используется для установки соединения и общения с кардридерным модулем по протоколу TCP/IP. Предоставляет свойства и методы для настройки адресов подключения и непосредственного подключения и отправки/приятия сообщений;
- `void Restart()` – объект-функция. Часть интерфейса взаимодействия с драйвером записи карты. Используется драйвером записи карты для перезагрузки кардридера. Используется крайне редко при возникновении

некоторых исключительных ситуаций. Заново вызывает функцию CardConnect для перенастройки и переподключения к карте;

- Str sAddress – объект-переменная. При загрузке драйвера в модуле обработки Chip Writer Driver в него передается необходимый адрес кардридера;
- str TCPSend(RStr data) – объект-функция. Используется для передачи данных в строке data по сетевым протоколам на кардридерный модуль с помощью объекта port. Переподключается к кардридерному модулю при потере соединения. Возвращает ответ кардридерного модуля на сообщение в виде строки текста;
- int Transfer(RStr sendData, LStr recvData) – объект-функция. Часть интерфейса взаимодействия с драйвером записи карты. Собирает строку текста с кодом для передачи сообщения sendData на карту и отправляет его посредством функции TCPSend. После отправляет дополнительную команду для считывания ответа карты, который записывается в recvData. Параметр recvData в данном случае аналогичен ссылкам в языке программирования C/C++.

Представленный список содержит в себе все объекты дерева кардридерного драйвера без упущений. Теперь следует подробнее рассмотреть внутреннюю структуру функций, которые и определяют механизмы работы драйвера. Последовательность, в которой будут рассмотрены функции будет определена нормальным порядком работы драйвера. Начнем с объекта-функции void Open(RStr address). Листинг кода объекта представлен в приложении А. Первой строчкой мы можем видеть директиву препроцессора #define, в которой записано определение port. Внутри FloraWare директивы работают таким же образом, как и в языке C/C++, но только в пределах кода отдельных объектов-функций. Здесь она используется для облегчения обращения к объекту port. Для обращения к любому объекту в FloraWare необходимо использовать его адрес, который можно получить путем заключения расположения объекта в символы «`». Так как обращений к объекту port достаточно много, для получения адреса было

заведено определение. Далее из параметра `address` вычленяются составляющие его адрес кардридера и тип интерфейса записи карты. Разделяются они в строке символом «-», поэтому расположение символа определяется встроенным для типа `Str` методом `Find`. Он определяет номер символа в строке и возвращает его в переменную `delimiterPos`. Стоит упомянуть, что индексы элементов в строках, массивах и других составных типов данных полностью соответствуют их номерам и начинаются с единицы. Если `Find` не найдет символ в строке, то он вернет число ноль. Далее проверяется присутствие символа. Если он имеется в строке, то в объекты `sAddress` и `interfaceType` записываются подстроки строки `address` с помощью встроенных функций `SubStr`. В случае с `sAddress` берется подстрока слева от символа-разделителя, то есть от символа с номером 1 до символа `delimiterPos - 1`, т.е. символа непосредственно слева от разделителя, а в случае с `interfaceType` берется вся оставшаяся часть строки кроме разделителя, т.е. от символа `delimiterPos + 1` до конца строки. При условии, если символ не был найден, это означает, что в строке `address` присутствует только адрес ридера, поэтому она сразу записывается в объект `sAddress`, а `interfaceType` по стандарту выставляется на значение «Contact». В завершение программа проверяет наличие соединения по сетевому каналу, если оно отсутствует, то она записывает настройки в объект `port` через его свойства, а именно свойство `Protocol`, указывающее сетевой протокол, записывается из встроенного набора констант `NetProtocol` значением константы `TCP` и свойство `Address` записывается значением объекта `sAddress`. После установки настроек вызывается методы `Start`, инициализирующий внутренние механизмы объекта, и `Connect`, осуществляющий подключение с таймаутом в секундах.

Далее рассмотрим листинги функций `WaitCard`, `Restart` и `CardConnect` (см приложение Б). Так как функции `WaitCard` и `Restart` просто вызывают функцию `CardConnect`, то не имеет смысла рассматривать все эти функции отдельно друг от друга. Так как ожидание карты происходит после инициализации кардридера и передачи настроек, то для ожидания карты внутри функции `WaitCard` достаточно вызова функции `CardConnect` с

передачей необходимого параметра из объекта `interfaceType`. То же самое можно сказать и про функцию `Restart`. Вследствие того, что функция `CardConnect` уже выполняет все необходимые для функции `Restart` действия, ее использования достаточно для реализации функционала объекта.

Теперь перейдем к рассмотрению функции `CardConnect`. При детальном анализе листинга функции можно отметить, что кардридерному модулю посредством функции `TCPSend` передается две отдельные команды. Подробнее проанализируем составляющие этих команд. Стоит упомянуть, что последним символом в тексте каждой команды явно указан символ с кодом 0 или же соответствующая ему управляющая последовательность «\0». Так как управляющая плата считывает все команды до представленного символа и только потом начинает анализ и выполнение программы, то его наличие в конце обособленного набора команд является обязательным условием для передачи на управляющую плату, а иначе плата будет ожидать символа и не выполнит программу до тех пор пока его не получит. Из соображений обеспечения читаемости рассматриваемых строк с командами символы «\0» будут пропущены. Внутри строки текста первой команды можно наблюдать следующий код: «`"".PrintMode(1);`». Данная команда обращается к текущему кардридеру с помощью пустых кавычек и выполняет для него метод `PrintMode`, который задает режим работы кардридера. Режимы задаются отдельными битами. Первый бит выставляет режим записи, второй – режим чтения. Единица, переданная как параметр метода, обозначает установку режима записи карты. Так как в считывании данных с карты необходимости нет, то второй бит не выставляется.

Рассмотрим следующий набор команд. Можно заметить, что второй набор команд собирается из нескольких строк текста. Заводится переменная `cardInit`, в которую внутри конструкции `switch` записываются команды для инициализации карты и передачи настроек в кардридер. Как видно из листинга кода функции `CardConnect`, набор команд записывается в переменную только в случае использования контактного интерфейса записи карты, т.е. если в параметре `interfaceType` содержится строка «`Contact`», остальные ситуации

являются исключительными, так как если кардридерный модуль будет использоваться как бесконтактный интерфейс (параметр `interfaceType` содержит строку «Contactless»), карта не сможет быть записанной. На текущий момент кардридерные модули эмбоссеров MATICA не поддерживают такой стандарт записи чипов карт, а именно стандарт ISO14443-A, поэтому такой вариант записи просто не представляется возможным. Рассмотрим строку со второй по счету передаваемой в кардридерный модуль собранной командой при условии выбора контактного интерфейса записи карты: «`my_card:=0; where(){ flow.Throw(0, ""); my_card = card(\"SC1\", \"ISO7816-3\", 1); my_card.VccDef (3000); my_card.FreqXSet(5000); my_card.Reset().Hex }`». Первый оператор «`:=`» создает в блоке оперативной памяти, принадлежащему используемому кардридеру, новую переменную `my_card` и записывает в нее целое число ноль. Так как язык TCPScript является динамически-типизированным, то не имеет значения какие данные будут записаны в переменную на этапе создания. Далее используется конструкция `where`, которая указывает, что в принадлежащем ей блоке кода будет происходить обращения к данным из блока оперативной памяти используемого на данный момент кардридера. Внутри блока кода конструкции первой инструкцией прописана инструкция для проверки установленного режима работы кардридера. Если в режиме работы не будут выставлены биты, а именно режим будет содержать значение ноль, то программой будет выброшена ошибка, которая прервет выполнение текущего набора команд, а также обработку внутри `TranzWare CardFactory`. Далее, в созданную переменную `my_card` записывается объект `card`, в конструкторе которого указаны параметры:

- 1) “SC1” – внутреннее название, с которым инициализируется карта. Сокращение от «Smart Card 1»;
- 2) “ISO7816-3” – название контактного стандарта записи карты. Кардридерные модули эмбоссера поддерживают единственный, представленный, стандарт. Параметр введен больше для использования в будущих модулях, в которых планируется поддержка других стандартов;

3) Последний параметр указывает на количество кардридеров, которые инициализирует объект, отсчитывая от текущего и включая его, и с которыми он будет производить работу. Так как работа осуществляется с единственным кардридером, то в параметре указана цифра один.

При создании объекта `card`, кардридерный модуль проверяет наличие карты в кардридере, подавая небольшое напряжение на контакты кардридера. Если карта не была найдена в кардридере, то конструктор объекта заблокирует выполнение программы до тех пор, пока карта не будет вставлена в кардридер и контакты кардридера не прижмутся к контактной площадке чипа. Далее указываются напряжение, которое будет подаваться с кардридера на карту и частота опроса карты методами `VccDef` и `FreqXSet` соответственно. Стоит упомянуть, что параметры данных методов указываются в милливольтках для `VccDef` и герцах для `FreqXSet`. Так как чипы карт достаточно чувствительные к небольшим разностям в напряжении, необходимо точно выставить напряжение, чтобы в дальнейшем не было проблем с записью карт. Конечно, большинство карт работает при напряжении на контактах в 3 вольта, поэтому это значение и было указано в методе. Говоря о частоте опроса, для всех современных карт с чипами такая частота в соответствии со стандартами должна быть 5 кГц, что и было указано в методе установки частоты.

Последняя команда `Reset`, используемая в качестве метода `my_card`, перезагружает кардридер и при этом подгружает все заданные ранее настройки. Кроме того, `Reset` опрашивает карту и получает ее ATR, который атрибутом `Hex` переводится в шестнадцатеричную запись. ATR представляет из себя шестнадцатеричную запись, в которой записано расположение всех блоков памяти чипа карты. Не смотря на то, что в драйвере ATR не используется, он все равно записывается в отдельно созданную для этого переменную с соответствующим названием. Сделано это по рекомендации от сотрудников компании «Компас плюс» для возможного использования в будущих версиях драйвера. Обычно ATR получается не от карты напрямую, а из спецификаций производителей карт, так как это удобнее и на основе этого разрабатываются все драйвера записи карты.

Далее рассмотрим ключевые функции, посредством которых и осуществляется взаимодействие с кардридерным модулем и передача данных на карту. Речь идет о функциях TCPSend и Transfer. Листинги кодов данных функций представлены в приложении В. Рассмотрим для начала код функции Transfer, которая, как было упомянуто выше, является частью интерфейса взаимодействия с драйвером записи карты. Для записи набора команд создается переменная req. В нее записывается следующая строка с командами: «where(){ res := \"\"; res = my_card.Apdu(\"\" + StrToHex(sendData, false, true) + \"\").Hex }». Как можно наблюдать, используется та же конструкция where для получения доступа к созданной переменной my_card в блоке оперативной памяти кардридера. Кроме того, создается новая переменная res, в которую записывается результат выполнения метода Apdu для my_card, преобразованный в шестнадцатеричный вид атрибутом Hex. Метод Apdu передает шестнадцатеричную команду на карту и возвращает ответ карты в двоичном представлении. Так как все данные при работе с чипами карт представляются в шестнадцатеричном формате, то результат преобразовывается. Готовый преобразованный результат записи получается в виде ответа карты. Он записывается в переменную res уже в коде функции Transfer. Содержимое переменной res уже записывается в переданный параметр-ссылку rescvData для дальнейшей обработки в коде драйвера записи карты, который будет вызывать функцию Transfer. Кроме того, функция Transfer обязательно должна возвращать целое число. Для этого карта отдельно опрашивается для получения последнего ответа с помощью метода атрибута Sw. Результат также преобразовывается к шестнадцатеричному виду и получается в качестве результата. После, ответ карты преобразуется к обыкновенному десятичному целому числу и возвращается из функции.

Функция TCPSend применяется только для обеспечения сетевого взаимодействия драйвера с кардридерным модулем. Во-первых, с помощью директив препроцессора вводятся определения для времени ожидания и таймаута подключения и передачи данных. Так же, как это было и в функции Open, сначала проверяется наличие подключения к кардридерному модулю.

Если оно отсутствует, объект `port` заново настраивается и в коде производится попытка повторного подключения. Далее вызывается метод `Send` объекта `port`, в который передаются необходимые данные, как полученный строковый параметр функции `TCPSend` – `data`, и таймаут, по которому стоит выбросить ошибку в случае, если не придет сообщение о принятии данных от кардридерного модуля. К сведению, таймаут указывается в секундах. После успешной отправки сообщения, запускается цикл ожидания, условиями выхода которого являются или появление данных от кардридерного модуля, что можно проверить атрибутом объекта `port` – `RxReady`, а также достижение таймаута, который отсчитывается специальной переменной `counter`. При этом между каждыми двумя проверками происходит ожидание в одну десятую секунды процедурой `Sleep`, что задано определением `waitTime`. При достижении любого из установленных условий происходит считывание полученных данных методом `Recv` объекта `port` в строковую переменную `res`. Параметрами метода являются ссылка на переменную, в которую будут записаны считанные данные, максимальное считываемое количество символов и таймаут. Максимальное количество символов указано достаточно большим числом для обеспечения возможности получения ответов различной длины. Таймаут же указан нулем, так как ожидание данных от кардридерного модуля уже производилось раньше, в цикле `while`, и повторно осуществлять ожидание нет смысла. Если функция `Recv` не сможет считать данные, то она выбросит ошибку, которая прервет выполнение обработки. После успешного принятия сообщения от кардридерного модуля, необходимо проверить наличие ошибок, которые могли возникнуть при обработке переданных команд. В случае возникновения ошибки при выполнении переданной программы, в полученной от кардридерного модуля строке будет обязательно содержаться подстрока «`exception`», как это указано в документации к языку программирования `TCPScript`, поэтому проверки наличия подстроки с помощью метода строковых переменных `Find` будет достаточно для определения наличия ошибки. Если ошибка будет найдена программа обязательно выкинет исключение и прервет выполнение обработки, так как

при возникновении любых ошибок в кардридерном модуле небезопасно продолжать с ним работу.

Опишем последние две функции, необходимые для работы с кардридерным модулем, а именно функции EjectCard и CardClose. Подробно рассмотрим листинги кодов этих функций (см приложение Г). Функция EjectCard всего-лишь использует отдельно созданную функцию отключения кардридера и закрытия подключения к карте CardClose по аналогии с функциями WaitCard и Restart, рассмотренными выше. Кроме того, EjectCard отлавливает ошибки, которые могут возникнуть в функции CardClose с помощью конструкции try-catch. Так как процесс отключения кардридера и очищения настроек является необязательным условием для продолжения обработки карты, то лучше всего не прерывать процесс обработки в случае возникновения ошибок на этом этапе, а просто написать в трассу персонализации сообщение о возникновении ошибки, чтобы возможно было зафиксировать факт ее возникновения. Функция CardClose же содержит небольшой программный код, отправляющий единственный набор команд на кардридерный модуль. Такой набор состоит из двух команд, представленных методами Off и Close переменной my_card. Первая команда выключает кардридер и очищает его настройки, вторая – полностью очищает созданный объект card и освобождает выделенную под него оперативную память. Такая последовательность действий при завершении обработки карты с использованием кардридерного модуля четко регламентирована в предоставленной компанией MATICA документацией.

3.2 Порядок работы с разработанным кардридерным драйвером

После полного описания структуры драйвера нам не могут открыться способы работы с этим драйвером и порядок его использования как части программного продукта TranzWare CardFactory, поэтому данный подраздел направлен на освящение этих моментов для получения полного представления места разработанного драйвера в такой большой программной системе. В данном подразделе будет описан процесс настройки и обработки пластиковой банковской карты средствами разработанного кардридерного драйвера с

использованием кардридерного модуля эмбоссера MATICA «с нуля». В каждом пункте будет подробно описан отдельный этап, необходимый для осуществления обработки.

3.2.1 Порядок настройки карточного продукта

Любая обработка банковской карты средствами TranzWare CardFactory всегда начинается с этапа создания и настройки карточного продукта внутри приложения. Как было упомянуто в первом разделе, карточный продукт – набор настроек этапов обработки карты, который впоследствии используется для осуществления непосредственной обработки внутри программной системы. В данном пункте будет описан процесс создания и настройки нового карточного продукта для обработки пластиковой карты средствами разработанного кардридерного драйвера.

Начать следует с создания нового карточного продукта. В CardFactory все карточные продукты создаются в разделе настроек. В нем присутствует несколько разделов, в которых возможно создать новый продукт, а именно в разделах CardProducts и Issuers->*Issuer_name*. Последний указанный раздел представляет из себя «производителя карт» внутри CardFactory. Каждый создаваемый продукт обязательно должен принадлежать какому-либо производителю в пределах продукта. Производителя так же можно создать в разделе Issuers. Производители служат только для объединения карточных продуктов под одним разделом, поэтому условимся, что уже создан необходимый производитель под названием «Konstantin». Перейдя в раздел созданного производителя возможно создать в его владении новый карточный продукт, нажав кнопку на верхней панели инструментов с пиктограммной зеленого плюса. При этом в списке продуктов, который отображается посередине окна, появится новый пустой элемент с тремя полями для ввода. В первом поле необходимо выбрать тип карточного продукта. Он используется чаще всего для категоризации продуктов в пределах CardFactory по платежным системам или производителям пластиковых карт. Значение поля никак не влияет на процесс записи карты, поэтому опустим процесс создания категории и выберем заранее заготовленную категорию «MIR», так как

обрабатываемая карта будет принадлежать именно этой платежной системе. Остальные два поля используются для записи названия будущего карточного продукта и комментария к нему. Название и комментарий стоит указывать таким образом, чтобы всем сотрудникам, которые работают с программной системой CardFactory было предельно ясно для какого типа карт производилась настройка, а также, при необходимости, для какого типа эмбоссера и/или кардридера внесены настройки. Таким образом, названием продукта будет «CASTLE_MIR_NSPK MATICA» для указания типа карты и используемого эмбоссера. В комментарии же запишем «MATICA embosser CM only», для разъяснения, что в данном продукте не настроена полноценная обработка карты с помощью эмбоссера MATICA, а настроено только взаимодействие с кардридерным модулем. После записи всех полей для применения изменений необходимо нажать кнопку на верхней панели инструментов с пиктограммой зеленой галочки, вследствие чего новый карточный продукт будет создан и записан в базу данных CardFactory и для него в разделе производителя «Konstantin» появится отдельный раздел.

Перейдем к этапу добавления модулей в созданный карточный продукт. Перейдя в раздел продукта и его подраздел под названием «Modules», можно увидеть, что в средней части интерфейса присутствует поле для пока что пустого списка модулей продукта, а также четыре кнопки в верхней части. Для начала необходимо добавить один из сервисных модулей в карточный продукт. Такой модуль будет отвечать за все криптографические операции в процессе обработки карты, поэтому иметь хотя бы один необходимо для полноценной обработки. Добавим сервисный модуль «Software Security Module», он будет использовать внутренние средства CardFactory для криптографических операций, в отличие от других сервисных модулей, которые в основном полагаются на криптографические сервера. Для тестового сценария такого сервисного модуля будет вполне достаточно. Далее внесем в список «рабочие» модули, которые непосредственно будут осуществлять этапы обработки пластиковой карты. Стоит упомянуть, что и сервисные и рабочие модули добавляются в список посредством нажатия кнопок с

соответствующими названиями в верхней части интерфейса. При нажатии любой из них, появляется окно со списком всех еще не добавленных сервисных или рабочих модулей в карточный продукт в зависимости от нажатой кнопки. Вернемся к рабочим модулям, список необходимых для полноценной обработки модулей уже был представлен в первой главе, но для лучшего восприятия повторим его здесь с кратким описанием назначения каждого модуля. Добавим в продукт следующий набор модулей для обработки:

- File Loader – для загрузки данных обрабатываемых карт из файлов;
- PIN Generator & Printer – для генерации PIN карты;
- Attributes filler – для заполнения атрибутов карты, таких как сгенерированный PIN, пользовательские поля карты, Service code, MBR, PVKI и т.д. Также может использоваться для генерации PAN карты;
- CVV Calculator – для расчета значения, используемого для подтверждения валидности данных карты (CVV/CVC – Card Verification Value/Card Verification Code) на основании специальных ключей CVK;
- PVV Calculator – для расчета значения, используемого для подтверждения PIN карты (PVV – Pin Verification Value) на основании специальных ключей PVK;
- EMV Preparator – для подготовки данных, записываемых на чип карты. Обычно это специальные ключи, параметры карты, флаги, отвечающие за определение функциональности карты, код валюты и т.д.;
- ODBC Writer – для записи настроенных данных в определенную базу данных;
- Chip Writer Driver – для записи данных на чип карты и настройки процесса.

Так как нас не сильно интересуют подробности процесса настройки модулей, кратко опишем его для каждого модуля ради лучшего понимания процесса в целом. Начнем с первого в списке модуля – File Loader. Для его правильной настройки необходимо понимать, какие данные необходимо загружать из файла для обработки карты, а какие данные будут указаны в

других модулях. В представленной ситуации, такие данные как Service code, MBR, PAN будут указаны в полях других модулей, поэтому в модуле File Loader во вкладке Template settings выставлены следующие поля для загрузки из файла, которые не могут быть указаны в других модулях, а именно имя клиента и телефон. Кроме того, в этой же вкладке возможно указать символ-разделитель для считываемых из файла полей. Такой символ был указан как символ двоеточия для удобства. Так как это тестовый сценарий, в основной вкладке этого модуля был указан способ загрузки файла «по выбору», то есть файл будет выбираться пользователем при запуске обработки в отдельном окне из списка доступных в директории поиска. Кроме того, была указана сама директория для поиска файлов, как директория, специально созданная в папке Documents под названием CardData. В этой директории был создан файл MaticaTest.txt, в котором был прописан следующий текст: «KONSTANTIN;+79999999999». Опять же, так как это тестовый сценарий, сами данные не так сильно волнуют нас, как то, верно ли они обработаются другими модулями и запишутся на карту.

Настроек для модуля PIN Generator & Printer не так много. Так как нам необходимо сгенерировать PIN для карты, мыставляем галочку с соответствующим названием в интерфейсе модуля настроек. Кроме того, обязательно нужно указать один из добавленных сервисных модулей, который и будет генерировать PIN. В нашем случае указываем единственный возможный вариант – Software Cryptographic module.

Перейдем к модулю Attributes Filler. В интерфейсе модуля возможно указать такие необходимые для обработки карты данные, как Service code, MBR, PAN и PIN. Так как PIN будет генерироваться в предыдущем модуле, поле с ним останется пустым. Service code и MBR должны быть указаны как числа 100 и 201 соответственно в соответствии со спецификациями платежной системы MIR. В модуле Attributes Filler присутствует возможность генерации PAN карты. Этот вариант обязательно будет использоваться, поэтому выставляется соответствующая галочка в интерфейсе модуля настроек. К тому

же, так как PAN генерируется, необходимо указать сервисный модуль в поле в верхней части интерфейса модуля настроек.

В модулях PVV Calculator и CVV Calculator настройка проводится схожим образом. В обоих модулях необходимо выставить ключи для расчета PVV и CVV соответственно. Они выставляются в отдельных полях и их можно указать несколькими способами. Можно вписать ключ в шестнадцатиричной кодировке вручную или же воспользоваться функцией администрирования ключей в CardFactory. Так как это тестовый случай, воспользуемся последним вариантом в силу его простоты и высокой скорости настройки. Функция администрирования ключей представляет из себя окно с вариантами получения ключей из различных источников, таких как файлы, криптографические сервера, сертифицирующие агентства и т.д. Так как производственные ключи в наличии отсутствуют, то сгенерируем новый ключ через окно. После того, как ключи были сгенерированы в обоих модулях, необходимо также указать сервисный модуль, использующийся для внутренних расчетов на этапе обработки. Естественно, выбран единственный доступный вариант.

Перейдем к модулю EMV Preparator. Фактически, этот модуль является самым масштабным по количеству выставляемых параметров и настроек среди других модулей. В этом модуле под контроль ставится каждый параметр, который будет впоследствии записан на чип карты и который будет непосредственно влиять на ее функциональность. Внутри интерфейса модуля возможно выставить такие настройки, как вид платежной системы, код используемой валюты, вид платежной карты(кредитная/дебетовая) и т.д. Настроек в модуле огромное количество, поэтому, чтобы не останавливаться надолго на этапе настройки данного модуля, все необходимые настройки были скопированы из одного из уже настроенных карточных продуктов под карты того же производителя.

Перейдем к предпоследнему модулю, необходимому для обработки карты – ODBC Writer. Как было сказано выше данный модуль используется для записи данных карты в базу данных по выбору пользователя. Настройка

данного модуля будет коротка, необходимо всего-лишь указать базу данных и таблицу, в которую будут записаны данные. Из соображений совместимости между CardFactory и Maticard, а также скорости подготовки среды для обработки была выбрана база данных в виде файла MS Access. Так как оба приложения для обработки имеют возможность работы с такой базой, и кроме того нет необходимости разворачивать сервер для записи данных в эту базу, это стало идеальным вариантом для представленного сценария. В модуле настроек был выставлен тип базы, как MS Access, выбран созданный заранее файл MaticaDB.access, а также выбрана заранее созданная в файле таблица MaticaCardData.

В завершение подробно разберем настройку модуля Chip Writer Driver, так как именно этот модуль обработки будет использоваться для записи чипа пластиковой карты через кардридерный модуль эмбоссера MATICA и именно в этом модуле настроек были внесены доработки в интерфейс для добавления возможности настройки разработанного кардридерного драйвера. Так как некоторые данные для передачи на карту подлежат шифрованию, то в данном модуле, как и в остальных, необходимо указать сервисный модуль для управления данной операцией. Такой модуль в продукте единственный, поэтому указываем именно его. После, необходимо указать контактный и/или бесконтактный интерфейс для записи карт. Вследствии того, что кардридерные модули эмбоссера поддерживают только контактный интерфейс записи, мы указываем именно его. Для представленного тестового сценария мы будем осуществлять запись единственной пластиковой карты средствами кардридерного модуля, поэтому укажем контактный интерфейс для однопоточного режима записи, а именно «Embosser via Smartware interface». Файл обмена указывается внутри рамки с надписью «Embosser Settings». Так как разработанный кардридерный драйвер не является первым программным решением внутри CardFactory, осуществляющим общение с внешними утилитами для обработки карт с помощью эмбоссеров через файлы обмена, то такая настройка уже присутствует внутри Chip Writer Driver. В описанной рамке есть таблица используемых кардридеров эмбоссера.

Необходимо добавить в нее новый кардридер кнопкой с пиктограммой черного плюса в верхней части рамки. После этого, в появившейся строке таблицы, в поле первого столбца можно открыть выпадающий список и выбрать нужный ридер. Выбираем вариант «SmartWare Contact Reader 1», так как в представленном сценарии для записи карты используется контактный кардридер МАТІСА под номером один в кардридерном модуле. В поле второго столбца указываем путь к созданному файлу обмена. В представленном сценарии файл создан в папке документы под рабочим пользователем под названием «Exchange.xch».

После, двигаясь ниже по интерфейсу модуля настроек можно увидеть добавленную рамку с надписью «Settings via Smartware». В рамке находится три элемента: два поля для ввода с надписями «Machine IP-base» и «Port» соответственно и галочка с надписью «Use TCP». Так как мы будем использовать добавленный режим записи через кардридерный модуль по сети, то выставляем галочку и два имеющихся поля ввода становятся активными. Прописываем в первое поле для ввода адрес «192.168.0.80», так как в соответствии с документацией к предоставленному кардридерному модулю, данный адрес служит для него базовым. Во второе поле прописываем порт «5050», также в соответствии с предоставленной документацией. В завершение, чтобы модуль обработки Chip Writer Driver вообще смог получить доступ к карте, необходимо указать криптографический мастер-ключ карты. Его можно получить из спецификации производителя карты, что и было сделано.

После всех приведенных в данном подразделе действий по настройке карточного продукта, его можно считать полностью настроенным и готовым к запуску обработки. Большинство настроек модулей карточного продукта выставляется в соответствии с необходимой документацией, спецификацией или другими нормативными документами, регламентирующими необходимые для тех или иных типов платежных систем или выпускаемых карт настройки. Некоторые же настройки при производстве карт банками, выставляются в соответствии с требованиями этого банка. Будь то реальный выпуск карт или

тестовые сценарии, в любом случае при настройке необходимо пользоваться необходимыми документами, содержащими все нужные данные, а иначе, при дальнейшей обработке карты возможно возникновение исключительных ситуаций и ошибок, которые не позволят полноценно завершить обработку карточного продукта.

3.2.2 Порядок запуска обработки

После успешной настройки карточного продукта можно приступить к подготовке запуска обработки пластиковой карты, а также произвести непосредственный запуск и осуществить обработку карты. Для успешного запуска необходимо подготовить среду и запустить все необходимые средства, которые будут использоваться в процессе обработки. В данном пункте будет описан весь порядок запуска обработки для рассматриваемого тестового сценария с использованием настроенного в прошлом подразделе карточного продукта в среде CardFactory.

Начнем с подготовки среды для осуществления обработки. Единственным средством, кроме, собственно, TranzWare CardFactory, которое необходимо для обработки пластиковой карты средствами разработанного кардридерного модуля с использованием кардридерного модуля эмбоссера MATICA, является утилита Maticard PRO, поставляющаяся компанией MATICA вместе с ее эмбоссерами для осуществления эмбоширования карт. Нужно запустить утилиту и настроить в ней новый «карточный продукт» по аналогии с CardFactory, только в данном случае настройке будет подлежать единственный модуль, модуль записи чипа карты. Аналогом карточного продукта внутри Maticard является работа (job). Создавая новую работу, возможно выбрать базу данных и таблицу, из которых будут выгружаться данные для обработки карт. Выберем для выгрузки созданную базу MS Access и таблицу в ней как это было указано в прошлом подразделе. Кроме этого, при создании работы в последней вкладке окна добавляем модуль Chip Writer и переходим внутрь модуля двойным нажатием по нему. Внутри модуля нас интересует только одна настройка «External DLL». В ней указываем расположение файла с внешней библиотекой DLL, которая будет

использоваться для передачи управления этапом записи карты на CardFactory. На этом настройка работы может быть окончена.

Так как других программных средств для осуществления обработки, кроме TranzWare CardFactory и Maticard PRO, не требуется, то приступим к описанию действий, необходимых для подготовки аппаратной составляющей. Во первых необходимо собрать, если он находится в разобранном состоянии, кардридерный модуль эмбоссера MATICA, то есть необходимо подключить блок питания к управляющей плате посредством имеющегося коннектора, а также проверить и подключить все имеющиеся кардридеры в расположенные на плате сокет для них. После, необходимо подключить блок питания кардридерного модуля в розетку и визуально оценить состояние модуля. Если никаких очевидных проблем не выявлено и все светодиодные лампочки на плате загорелись, то можно подключить модуль в сетевую плату используемой ЭВМ через кабель Ethernet напрямую. Обязательно после этого нужно проверить наличие модуля в сетевых подключения ЭВМ. При использовании ОС Windows, как в представленном сценарии, это можно сделать перейдя в панель управления в раздел «Центр управления сетями и общим доступом», а в нем уже перейти в раздел «Изменение параметров адаптера». В появившемся окне, в списке должен отобразиться подключенный модуль, помимо других сетевых подключений. После этого карта вставляется в кардридер. При этом обязательно нужно вставить ее правильными концом и стороной. Вставляться карта будет в первый по счету кардридер, поэтому нужно взять первый слева кардридер и посмотреть внутрь. Оценив местонахождение прижимных контактов в кардридере, нужно вставить карту в кардридер чипом по направлению к контактам. Так как карта вставляется вручную нужно разогнуть две соединенные пружиной половины и установить карту так, чтобы все контакты оказались прижатыми к нужным местам на карте.

После настройки и проверки всех аппаратных и программных средств, можно приступать к обработке пластиковой карты. Для этого вначале запускается обработка внутри TranzWare CardFactory. Перейдя в раздел Processing в подраздел Batches, создаем новый пакет (batch) карт для

обработки и переходим в появившийся для него раздел. В интерфейсе пакета выбираем производителя «Konstantin», созданную категорию карт «MIR» и созданный карточный продукт по названию в отдельных полях. После этого нажимаем кнопку с надписью «Start» в левом верхнем углу интерфейса и обработка будет запущена. Практически сразу откроется окно со списком всех файлов в настроенной директории модуля File Loader. Выбираем из списка необходимый файл двойным нажатием левой кнопки мыши и нажимаем в окне кнопку «Continue». Во время обработки мы можем видеть, как в логах обработки отображаются сообщения о запуске и завершении работы каждого модуля настроенного карточного продукта. Если мы хотим получить более подробную информацию о производимых в модулях обработки действиях, то можно открыть трассу персонализации кнопкой «Trace» в верхней части интерфейса. При этом откроется окно, в котором в режиме реального времени будут отображаться все логируемые сообщения модулей обработки. Обычно именно по трассе персонализации выявляют причины различных ошибок и проблем в процессе обработки. Когда обработка дойдет до модуля Chip Writer Driver, он будет ожидать xml-сообщения в файле обмена. До этого момента обработка будет полностью остановлена и перейдет в режим ожидания.

Только после того, как обработка в CardFactory остановится на модуле Chip Writer Driver, можно будет приступить к запуску обработки внутри утилиты Maticard PRO. В главном окне выбираем созданную работу и нажимаем кнопку в верхней части интерфейса с пиктограммой банковской карты и человека. Открывается новое окно, которое показывает сколько карт подлежит обработке. Если все операции до этого были выполнены правильно, то в поле с количеством карт должна отображаться цифра один. Нажимаем кнопку «Start» в правом нижнем углу. При этом происходит несколько вещей. Во-первых Maticard PRO открывает новое окно, отображающее статус работы. Во-вторых, внутри TranzWare CardFactory продолжается процесс обработки, что можно заметить по новым сообщениям в трассе персонализации.

Как только запись чипа будет завершена, CardFactory и Maticard должны завершить обработку, что в CardFactory можно увидеть по сообщению в логах,

а в Maticard – по статусу обработки работы в открывшемся окне. На этом процесс обработки пластиковой карты средствами разработанного кардридерного драйвера с использованием кардридерного модуля эмбоссера MATICA может считаться завершенным.

3.3 Выводы

По итогам данного раздела были рассмотрены результаты разработки и внедрения программного решения в продукт TranzWare CardFactory. Подробно были описаны:

- Архитектура разработанного драйвера, которая представляет из себя набор механизмов, обеспечивающих полноценную работу с кардридерным модулем эмбоссера MATICA через драйвер записи карты.
- Объектное дерево драйвера, состоящее из нескольких объектов-функций с кодом для взаимодействия с кардридерным модулем, некоторые из которых составляют интерфейс взаимодействия драйвера записи карты с разработанным кардридерным драйвером, а также нескольких вспомогательных объектов.
- Порядок настройки обработки внутри CardFactory, посредством создания нового карточного продукта, добавления в него всех необходимых модулей и внесения настроек в каждый из них в соответствии с необходимой документацией.
- Порядок запуска обработки, при котором используются как TranzWare CardFactory, так и Maticard PRO.

Описанные в данном разделе элементы помогут лучше понять процесс разработки и принятые в этом процессе технические решения. Кроме того данный раздел даст понимание неопытным пользователям как проходит процесс настройки и обработки карточного продукта внутри CardFactory, а также даст понимание того, как должна проводится настройка карточного продукта для использования внедренного драйвера.

ЗАКЛЮЧЕНИЕ

Не смотря на то, что банки по всему миру каждый день выпускают огромное количество пластиковых карт для своих клиентов, сам процесс выпуска не всегда хорошо оптимизирован. Конечно, процесс можно всегда доработать для удовлетворения требованиям банка к скорости выпуска, но некоторые принимают решение отложить или же вовсе не проводить доработку процесса. Тому могут быть разные причины, но основной все же является огромное количество средств, которое необходимо на внесение изменений. Кроме огромных трудозатрат, требуется большое количество времени на проектирование, разработку и тестирование будущих изменений. Эмбоссеры все же являются сложной многомодульной технической системой. В связи со сложностью и большим количеством компонентов такой системы, внесение любых изменений в процесс представляет большую сложность и требует глубокого исследования. Процесс доработки неизбежно сопряжен с появлением ошибок, так как происходит работа с аппаратными средствами эмбоссера и проблемы могут возникнуть как в используемом для управления им ПО, так и в аппаратной части.

Клиентами российской компании «Компас плюс» был подан запрос на разработку решения по прямому управлению кардридерным модулем эмбоссеров итальянской компании MATICA для программного продукта TranzWare CardFactory. Целью разработки являлась автоматизация записи данных на чип пластиковой карты средствами TranzWare CardFactory с использованием кардридерного модуля эмбоссера MATICA. Такое решение позволило бы клиентам объединить процессы эмбоссирования и записи чипа карты при использовании эмбоссеров MATICA в один непрерывный процесс персонализации, в ходе которого карта пройдет через все необходимые модули эмбоссера и на выходе будет получена готовая, обработанная, пластиковая карта.

В ходе исследования по части аппаратных компонентов работы были выявлены общие этапы работы эмбоссеров. Кроме того, для лучшего понимания всех этапов персонализации карты средствами эмбоссеров, была

изучена история появления платежных карт и сопутствующих способов записи информации на них. К тому же была подробно изучена документация к кардридерному модулю эмбоссера, предоставленная компанией MATICA, из чего были выявлены основные методы взаимодействия с модулем, основные составляющие кардридерного модуля, способы подключения к модулю и порядок работы с ним. Были подробно проанализированы и используемые программные средства. По результатам исследования была выявлена структура и внутреннее устройство программного продукта TranzWare CardFactory и платформы FloraWare, на базе которой он разработан. Поверхностно была рассмотрена и утилита Maticard PRO, поставляющаяся с эмбоссерами компанией MATICA. Не смотря на то, что никаких доработок в данном программном продукте не выполнялось, он был изучен вследствие того, что он необходим в процессе персонализации с использованием эмбоссеров компании MATICA. В дополнение были проанализированы текущие решения персонализации с использованием представленных эмбоссеров и имеющиеся кардридерные драйвера для того, чтобы подчеркнуть актуальность работы и ее необходимость, а также обеспечить информационную базу для дальнейших этапов проектирования, разработки и внедрения драйвера.

На этапе проектирования, на основании информации полученной на этапе исследования, были выявлены основные функциональные требования, предъявляемые к драйверу. Кроме того были спроектированы функциональные схемы, отражающие место драйвера в процессе обработки карты. Были полностью описаны все потоки данных и управления процессов обработки карты с использованием кардридерного драйвера. К тому же было спроектировано и описано дополнение к пользовательскому интерфейсу модуля Chip Writer Driver, призванному предоставить будущему пользователю возможность настройки параметров драйвера. В дополнение были подробно описаны все инструментальные средства, используемые для разработки в соответствии с поставленным техническим заданием.

После разработки готовое программное решение драйвера было внедрено в продукт TranzWare CardFactory. В работе были подробно рассмотрены и описаны все компоненты разработанного драйвера. Подробно были описаны все внутренние механизмы работы драйвера, а также его полное объектное дерево с подробным рассмотрением кодов функций. Кроме того был описан порядок работы в предложенном тестовом сценарии с внедренным драйвером, начиная с этапа настройки карточного продукта и заканчивая обработкой карты с использованием кардридерного модуля.

По итогам работы, поставленная цель была полностью выполнена. Программное решение драйвера прямого управления кардридерным модулем эмбоссера MATICA был внедрено в программный продукт TranzWare CardFactory для автоматизации записи чипа банковской пластиковой карты с использованием представленных эмбоссеров. Все поставленные задачи, от исследования до разработки и описания полученного программного решения, были в полной мере исполнены. На данный момент разработанное программное решение драйвера уже внесено в одну из рабочих версий TranzWare CardFactory, передано клиентам и используется в одном из процессинговых центров Нигерии для выпуска банковских карт с использованием эмбоссеров MATICA.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Платежные карты. Энциклопедия 1870–2006 / Павел Юржик; Пер. с чешск. — М.: Альпина Бизнес Букс, 2007. — 304 с. — ISBN 5-9614-0436-6
2. Объектно-ориентированный анализ и проектирование с примерами приложений на C++ / Гради Буч; Пер. с англ. под ред. И. Романовского и Ф. Андреева. - 2-е изд. - Москва : Vinom ; СПб. : Нев. диалект, 1998. - 558 с. : ил., табл.; 24 см.; ISBN 5-7989-0067-3
3. Унгер, А. Ю. Объектно-ориентированное программирование : учебник / А. Ю. Унгер. — Москва : РТУ МИРЭА, 2023. — ISBN 978-5-7339-2051-1.
4. Стаяно, Ф. Figma проектирование и прототипирование интерфейсов : руководство / Ф. Стаяно ; перевод с английского В. С. Яценкова. — Москва : ДМК Пресс, 2024. — 370 с. — ISBN 978-5-93700-302-7.
5. Остроух, А. В. Проектирование информационных систем : монография / А. В. Остроух, Н. Е. Суркова. — 2-е изд., стер. — Санкт-Петербург : Лань, 2021. — ISBN 978-5-8114-8377-8.
6. Компаниец, В. С. Проектирование и юзабилити-исследование пользовательских интерфейсов : учебное пособие / В. С. Компаниец. — Ростов-на-Дону : ЮФУ, 2020. — 107 с. — ISBN 978-5-9275-3637-5.
7. Лыгина, Н. И. Разработка требований к программному продукту : учебное пособие / Н. И. Лыгина, О. В. Лауферман. — Новосибирск : НГТУ, 2023. — 76 с. — ISBN 978-5-7782-4987-5.
8. Туманова, М. Б. Проектирование программных систем : учебное пособие / М. Б. Туманова, Е. К. Михайлова, Е. А. Муравьева. — Москва : РТУ МИРЭА, 2023. — ISBN 978-5-7339-2050-4.
9. Малышенко, Ю. В. Платежные карты, электронные платежные системы и их использование в торговле и таможенном деле : учебное пособие / Ю. В. Малышенко. — Санкт-Петербург : Интермедия, 2017. — 228 с. — ISBN 978-5-4383-0110-3.

ПРИЛОЖЕНИЕ А ЛИСТИНГ КОДА ОБЪЕКТА-ФУНКЦИИ OPEN

```
void Open(RStr address)
#define port `port`

int delimiterPos = address.Find("-");
if (delimiterPos) {
    *`sAddress` = SubStr(address, 1, delimiterPos - 1);
    *`interfaceType` = SubStr(address, delimiterPos + 1);
}
else {
    *`sAddress` = address;
    *`interfaceType` = "Contact";
}

if (!(port.Connected)) {
    port.Protocol = NetProtocol::TCP;
    port.Address = *`sAddress`;
    port.Start();
    port.Connect(30);
}
```

ПРИЛОЖЕНИЕ Б ЛИСТИНГИ КОДОВ ОБЪЕКТОВ-ФУНКЦИЙ WAITCARD, RESTART И CARDCONNECT

```
void WaitCard()
CardConnect(*`interfaceType`);

void Restart()
CardConnect(*`interfaceType`);

void CardConnect(str interfaceType)
str req, res;

req = "\"\".PrintMode(1);\0";
res = TCPSend(req);

str cardInit;
switch (interfaceType) {
    case "Contact":
        cardInit = "my_card = card(\"SC1\", \"ISO7816-3\", 1);my_card.VccDef
(3000);my_card.FreqXSet(5000);";
        break;
    case "Contactless":
        throw Ex::Error, 0, "Contactless personalization (ISO14443-A) not
supported now\nInvalid reader interface type: " + interfaceType;
        break;
    default:
        throw Ex::Error, 0, "Invalid reader interface type: " + interfaceType;
}
req = "my_card:=0;where(){flow.Throw(0, "");" + cardInit +
"my_card.Reset().Hex}\0";
str ATR = TCPSend(req);
```

ПРИЛОЖЕНИЕ В ЛИСТИНГИ КОДОВ ОБЪЕКТОВ-ФУНКЦИЙ TCPSSEND И TRANSFER

```
str TCPSend(RStr data)
#define iTimeout 5
#define waitTime 0.1

num counter = 0.0;

if (!(port.Connected)) {
    port.Protocol = NetProtocol::TCP;
    port.Address = *`sAddress`;
    port.Start();
    port.Connect(iTimeout);
}

port.Send(data, iTimeout);

while (!port.RxReady && counter < iTimeout) {
    Sleep(waitTime);
    counter += waitTime;
}

str res;
port.Recv(res, 100000, 0);

if (res.Find("exception"))
    throw Ex::Error, 0, res;

return res;

int Transfer(RStr sendData, LStr recvData)
```

```
    str req = "where(){ res := \"\";res = my_card.Apdu(\"\" + StrToHex(sendData,  
false, true) + \"\").Hex}\0\";  
    str res = TCPSend(req);  
    recvData = res;  
  
    req = "where(){my_card.Sw.Hex}\0\";  
    res = TCPSend(req);  
  
    int sw;  
    StrToInt(res, sw, 16);  
  
    return sw;
```

ПРИЛОЖЕНИЕ Г ЛИСТИНГИ КОДОВ ОБЪЕКТОВ-ФУНКЦИЙ EJECTCARD И CARDCLOSE

```
void EjectCard()
try {
    CardClose();
}
catch (...) {
    Trace(Severity::Warning, "Card close executed with error: " +
ExceptionMess());
}
```

```
void CardClose()
str req = "where(){my_card.Off();my_card.Close();}\0";
str res = TCPSend(req);
```